

```

/* =====
ACOLITE v2 - copilote de voyage dual-AI
✦ Gemini = tâches lourdes (destinations, itinéraires, budget...)
⚡ Groq = tâches simples déléguées (valise, phrases, infos, concierge)
Fichier unique · zéro backend · localStorage
===== */

/* =====
ÉCRAN DE DÉMARRAGE - placé TOUT EN HAUT, sans aucune dépendance.
Il se retire quoi qu'il arrive : si le reste du fichier plante,
l'erreur s'affiche à l'écran au lieu de bloquer sur le splash.
===== */

(function(){
  var boot = document.getElementById('boot');
  if(!boot) return;
  var bar = boot.querySelector('.boot-bar i');
  var lbl = document.getElementById('bootStep');
  var steps = [[18, 'Chargement des styles'], [42, 'Réveil du copilote'], [68, 'Connex
  var i = 0, dead = false;

  function hide(){
    if(dead) return;
    dead = true;
    boot.classList.add('gone');
    setTimeout(function(){ if(boot.parentNode) boot.parentNode.removeChild(boot);
  }
  function tick(){
    if(dead) return;
    if(i >= steps.length){ setTimeout(hide, 250); return; }
    var s = steps[i++];
    if(bar) bar.style.width = s[0] + '%';
    if(lbl) lbl.textContent = s[1];
    setTimeout(tick, 190);
  }
  tick();

  /* sécurité : jamais bloqué plus de 5 s */
  setTimeout(hide, 5000);

  /* si le script plante pendant le démarrage, on le DIT au lieu de rester figé *
  window.addEventListener('error', function(e){
    if(dead) return;
    if(lbl){
      lbl.style.color = '#FF6B00';

```

```

        lbl.style.fontSize = '.62rem';
        lbl.textContent = '⚠️ ' + (e.message || 'erreur au démarrage');
    }
    setTimeout(hide, 2500);
});
window.__acoliteBoot = { hide: hide };
})();

const LS_GEM   = 'acolite_gemini_key';
const LS_GEMM  = 'acolite_gem_model';    // modèle Gemini auto-détecté
const LS_GROQ  = 'acolite_groq_key';
const LS_GROQM = 'acolite_groq_model';
const LS_TP    = 'acolite_tp_token';
const LS_TRIP  = 'acolite_trip_v2';
/* Ordre de préférence – le premier dispo sur la clé sera utilisé */
const GEM_PREFERRED = ['gemini-2.5-flash', 'gemini-flash-latest', 'gemini-2.0-flash

const $  = s => document.querySelector(s);
const $$ = s => [...document.querySelectorAll(s)];
const esc = s => String(s ?? '').replace(/[\<>"]/g, c => ({'&': '&', '<': '<'}

let state = {
    step: 1,
    prefs: null,
    destinations: [],
    trip: null,
    mode: 'plane',
    cache: {},           // réponses IA
    checklist: {},       // valise cochée
    spends: [],          // dépenses réelles
    chatLog: [],         // concierge
    notes: '',           // carnet
    resas: [],           // réservations
    planAnswers: [],     // réponses aux questions du plan
    propAnswers: []      // réponses d'affinage des propositions
};

function save(){ localStorage.setItem(LS_TRIP, JSON.stringify(state)); }
function load(){
    try{ const s = JSON.parse(localStorage.getItem(LS_TRIP)); if(s) state = {...sta
}

let toastT;
function toast(msg){
    const t = $('#toast'); t.textContent = msg; t.classList.add('show');
    clearTimeout(toastT); toastT = setTimeout(()=>t.classList.remove('show'), 3200)
}

```

```

/* =====
ROUTEUR IA - heavy → Gemini · light → Groq (si clé) sinon Gemini
===== */
const CFG = window.ACOLITE_KEYS || {};
/* Backend Cloudflare : si configuré, les clés restent CÔTÉ SERVEUR.
Le navigateur n'en envoie aucune - c'est le mode recommandé en public. */
const API = () => (CFG.proxy || '').replace(/\/+$/, '');
const useBackend = () => !!API();
const gemKey = () => CFG.gemini || localStorage.getItem(LS_GEM) || '';
const groqKey = () => CFG.groq || localStorage.getItem(LS_GROQ) || '';
const groqModel = () => localStorage.getItem(LS_GROQM) || 'llama-3.3-70b-versatil';
const tpKey = () => CFG.travelpayouts || localStorage.getItem(LS_TP) || '';
const hasGroq = () => useBackend() || !!groqKey();

/* --- Découverte automatique du modèle Gemini disponible sur la clé --- */
async function resolveGemModel(key, force = false){
  /* Réglage "Modèle" du panneau Préférences */
  if(SET?.model && SET.model !== 'auto') return SET.model;
  if(!force){
    const cached = localStorage.getItem(LS_GEMM);
    if(cached) return cached;
  }
  const r = await fetch(useBackend()
    ? `${API()}/gemini/models`
    : `https://generativelanguage.googleapis.com/v1beta/models?key=${encodeURIComponent(
  if(!r.ok){
    const msg = await gemErrMsg(r);
    throw new Error('LIST:' + msg);
  }
  const d = await r.json();
  const names = (d.models || []).
    .filter(m => (m.supportedGenerationMethods || []).includes('generateContent'))
    .map(m => (m.name || '').replace('models/', ''));
  const pick = GEM_PREFERRED.find(p => names.includes(p))
    || names.find(n => n.includes('flash') && !n.includes('image') && !n.includes
    || names[0];
  if(!pick) throw new Error('LIST:Aucun modèle compatible sur cette clé.');
```

localStorage.setItem(LS_GEMM, pick);

return pick;

}

```

/* --- Message d'erreur lisible depuis la réponse Google --- */
async function gemErrMsg(r){
  let apiMsg = '';
  try{ const j = await r.json(); apiMsg = j.error?.message || ''; }catch(e){}
  if(r.status === 400 && /API key not valid|API_KEY_INVALID/i.test(apiMsg)) retur
```

```

    if(r.status === 403 && /SERVICE_DISABLED|has not been used|is disabled/i.test(a
    if(r.status === 403) return 'Accès refusé (403) – clé restreinte ? Vérifie les
    if(r.status === 429) return 'Quota gratuit atteint – attends 1 min ou passe sur
    if(r.status === 404) return 'Modèle introuvable (404)';
    if(r.status === 503) return 'Gemini surchargé (503), réessaie dans quelques sec
    return `Erreur ${r.status}${apiMsg ? ' - ' + apiMsg.slice(0,120) : ''}`;
}

```

```

async function gemini(prompt, expectJson = true, maxTok = 4096, _retry = false, t
    const key = gemKey();
    if(!key && !useBackend()){ toast('⚠ Clé Gemini absente de config.js'); throw n
    let model;
    try{
        model = await resolveGemModel(key);
    }catch(e){
        const m = String(e.message||'').replace(/^LIST:/,'') || 'Connexion à Gemini i
        toast('⚠ ' + m);
        throw new Error('BAD_KEY');
    }
    const body = {
        contents: [{ role:'user', parts:[{ text: prompt }] }],
        generationConfig: { temperature: temp, maxOutputTokens: maxTok }
    };
    if(expectJson) body.generationConfig.responseMimeType = 'application/json';
    const r = useBackend()
        ? await fetch(`${API()}/gemini`, {
            method:'POST',
            headers:{ 'Content-Type':'application/json' },
            body: JSON.stringify({ model, body }) /* aucune clé ne quitte le nav
        })
        : await fetch(`https://generativelanguage.googleapis.com/v1beta/models/${mode
            method:'POST', headers:{'Content-Type':'application/json'}, body: JSON.string
        });
    if(r.status === 404 && !_retry){
        /* modèle mis en cache devenu obsolète → re-détection puis retry */
        localStorage.removeItem(LS_GEMM);
        await resolveGemModel(key, true);
        return gemini(prompt, expectJson, maxTok, true);
    }
    if((r.status === 429 || r.status === 503) && !_retry){
        /* surcharge passagère → une seule nouvelle tentative après 1,6 s */
        await new Promise(res => setTimeout(res, 1600));
        return gemini(prompt, expectJson, maxTok, true, temp);
    }
    if(!r.ok){
        const msg = await gemErrMsg(r);
        toast('⚠ ' + msg);
    }

```

```

    if(r.status === 429) throw new Error('RATE');
    throw new Error('BAD_KEY');
  }
  const d = await r.json();
  let txt = (d.candidates?.[0]?.content?.parts || []).map(p=>p.text||'').join('')
  if(!txt) { toast('⚠ Réponse vide de Gemini, réessaie'); throw new Error('EMPTY') }
  if(!expectJson) return txt;
  txt = txt.replace(/```json|```/g,'').trim();
  return parseAI(txt);
}

```

```

async function groq(prompt, expectJson = true, maxTok = 2048){
  const body = {
    model: groqModel(),
    messages: [{ role:'user', content: prompt + (expectJson ? '\nRéponds UNIQUEMENT' : '') },
    temperature: 0.7,
    max_tokens: maxTok
  ];
  if(expectJson) body.response_format = { type:'json_object' };
  const r = useBackend()
    ? await fetch(`${API()}/groq`, {
        method:'POST',
        headers:{ 'Content-Type':'application/json' },
        body: JSON.stringify({ body }) /* la clé Groq reste sur le serveur */
      })
    : await fetch('https://api.groq.com/openai/v1/chat/completions',{
        method:'POST',
        headers:{ 'Content-Type':'application/json', 'Authorization':'Bearer ' + API_KEY },
        body: JSON.stringify(body)
      });
  if(r.status === 401){ toast('Clé Groq invalide – vérifie dans ⚙️'); throw new Error('GROQ_INVALID_KEY'); }
  if(r.status === 429){ throw new Error('GROQ_RATE'); }
  if(!r.ok) throw new Error('GROQ_HTTP ' + r.status);
  const d = await r.json();
  let txt = d.choices?.[0]?.message?.content || '';
  if(!expectJson) return txt;
  txt = txt.replace(/```json|```/g,'').trim();
  return parseAI(txt, false);
}

```

```

/* --- Auto-réparation JSON : récupère les réponses IA mal formées --- */
async function parseAI(txt, allowRepair = true){
  try{ return JSON.parse(txt); }catch(e){}
  const a = txt.indexOf('{'), b = txt.lastIndexOf('}');
  if(a > -1 && b > a){ try{ return JSON.parse(txt.slice(a, b + 1)); }catch(e){} }
  if(allowRepair && hasGroq()){
    try{

```

```

        const fixed = await groq('Ce JSON est invalide. Corrige-le sans changer son
        return JSON.parse(fixed.replace(/``json|``/g, ' ').trim());
    }catch(e){}
}
throw new Error('BAD_JSON');
}

/* ai('heavy'|'light', prompt) - retourne {data, via} */
async function ai(kind, prompt, expectJson = true, maxTok = 4096){
    if(kind === 'light' && hasGroq()){
        try{
            const data = await groq(prompt, expectJson, Math.min(maxTok, 4096));
            return { data, via:'groq' };
        }catch(e){
            // fallback silencieux sur Gemini si Groq plante (rate limit...)
            if(e.message === 'BAD_GROQ') throw e;
        }
    }
    const data = await gemini(prompt, expectJson, maxTok);
    return { data, via:'gemini' };
}

function loaderHTML(msg){ return `<div class="loader"><div class="orbit"></div>${
function errHTML(msg){ return `<div class="err">⚠️ ${esc(msg)}</div>`; }
function badge(via){ return via === 'groq' ? '<span class="ai-badge groq">⚡ Groq
}

/* ----- Contexte voyage ----- */
function ctx(){
    const p = state.prefs || {}, t = state.trip || {};
    let saison = '';
    if(p.depart){
        const mo = new Date(p.depart + 'T12:00:00');
        if(!isNaN(mo)) saison = mo.toLocaleDateString('fr-FR', { month:'long', year:'
    }
    return `Date d'aujourd'hui : ${new Date().toLocaleDateString('fr-FR')}.
    CONTEXTE VOYAGEUR :
    - Départ : ${p.from || 'non précisé'}${saison ? `\\n- Mois du voyage (calculé) : $
    - Destination choisie : ${t.nom || '?'}, ${t.pays || ''}
    - Durée : ${p.days || '?'} · Période : ${p.when || 'flexible'}
    - Budget/pers : ${p.budget || '?'} · Voyageurs : ${p.adults||2} adulte(s)${p.kids
    - Destination souhaitée : ${p.dest || 'libre, à proposer'}
    - Limites & conditions : ${p.free || 'aucune'}
    ${prefsBlock()}`;
}

/* =====
    ÉTAPE 1 - ENVIES → PROPOSITIONS (Gemini · heavy)

```

```

===== */
function readPrefs(extra){
  return {
    from: $('#fFrom').value.trim() || 'Paris',
    dest: $('#fDest').value.trim(),
    days: $('#fDays').value,
    when: $('#fWhen').value.trim(),
    budget:$('#fBudget').value,
    adults:+$('#fAdults').value || 2,
    kids:  +$('#fKids').value || 0,
    depart:$('#fDepart').value,
    free:  $('#fFree').value.trim().slice(0,600) + (extra ? ' | Affinage : ' + St
  };
}

async function proposeTrips(extra = '', lucky = false){
  const prefs = readPrefs(extra);
  state.prefs = prefs; save();
  const zone = $('#zoneResults');
  zone.innerHTML = `<div class="card">${loaderHTML(lucky ? "Roulette mondiale en
searchBar(true, lucky ? 'Roulette mondiale en cours... 🎰' : 'Acolite explore le
$('#btnGo').disabled = true; $('#btnLucky').disabled = true;

  const prompt = `Tu es Acolite, un expert voyage français, chaleureux et concret
${ctx()}}
${lucky ? 'MODE SURPRISE : propose des destinations inattendues, originales, auxq
TOUT DOIT ÊTRE TROUVÉ DÈS MAINTENANT : pour CHAQUE proposition, tu donnes déjà le

QUESTIONS DE PRÉCISION : si des infos te manquent pour viser juste (rythme, ambia

${(state.seen||[]).length && !prefs.dest ? 'DÉJÀ PROPOSÉ à ce voyageur (ne PAS re
${(getHistory())||[]).length ? 'VOYAGES DÉJÀ CHOISIS par ce voyageur par le passé
DÉCIDE toi-même du NOMBRE de propositions (1 à 3) selon la demande :
- La demande désigne UNE VILLE précise → UNE SEULE proposition : la meilleure for
- Un PAYS ou une région → 2 ou 3 villes/zones DIFFÉRENTES de ce pays.
- Demande ouverte → 3 destinations VRAIMENT différentes.
INTERDICTION ABSOLUE de proposer des voyages qui se ressemblent : chaque proposit
Respecte STRICTEMENT le budget, les limites, la durée et la période – attention à
RÈGLE ABSOLUE : uniquement des villes et lieux RÉELS. Budgets réalistes pour la s
${prefs.free && prefs.free.includes('Affinage') ? "Le voyageur a déjà répondu à d

Réponds UNIQUEMENT en JSON valide, structure exacte. Commence OBLIGATOIREMENT par
{
  "analyse":"3-4 phrases : profil du voyageur, contraintes clés (budget/saison/dis
  "destinations":[
    {
      "nom":"...", "pays":"...", "drapeau":"emoji drapeau",

```

```

    "resume": "2 phrases vendeuses et concrètes",
    "budget_estime": "ex: ~850€/pers tout compris",
    "duree_ideale": "ex: 5-7 jours",
    "meteo_periode": "ex: 26°C, ensoleillé",
    "points_forts": ["3 à 4 atouts courts"],
    "acces": "avion" ou "voiture" ou "train" ou "avion ou voiture",
    "iata": "code IATA aéroport principal, ex CDG, sinon null",
    "ville_aeroport": "nom ville aéroport le plus proche",
    "langue": "langue principale parlée",
    "monnaie": "monnaie locale",
    "transport_conseille": "avion" ou "train" ou "voiture",
    "transport_pourquoi": "6-10 mots : pourquoi ce transport vu le budget/conditi
    "transport_prix": "fourchette A/R par personne, ex 90-140€",
    "transport_duree": "durée porte-à-porte, ex 2h15 de vol + 1h de transferts",
    "logement_type": "1 ou 2 mots MAX : hôtel, appartement, auberge, villa...",
    "logement_quartier": "LE quartier précis conseillé (nom réel), ex Trastevere"
    "logement_prix": "fourchette par nuit, ex 80-120€",
    "logement_pourquoi": "6-10 mots : pourquoi ce quartier"
  }
],
"questions": [
  { "texte": "question courte et UTILE pour préciser le voyage", "options": ["3-4 ré
]
}
"questions" : 1 à 3 questions qui aideraient VRAIMENT à préciser le voyage (dates

try{
  const d = await gemini(prompt, true, 8192);
  state.destinations = d.destinations || [];
  state.seen = [...new Set([...(state.seen||[]), ...state.destinations.map(x=>x
  state.lastProps = d; save();
  renderDestinations(d);
  gotoStep(2);
  /* → Questions de précision AVANT que tu ne choisisses : la pop-up s'ouvre ic
  const qs = (d.questions || []).filter(q => q && q.texte);
  if(qs.length && !state._qsDone) openQsPopup(qs);
  else { $('#ovQs').classList.remove('show'); $('#zoneQs').innerHTML = ''; }
}catch(e){
  if(e.message !== 'NO_KEY') zone.innerHTML = `<div class="card">${errHTML("Imp
  else zone.innerHTML = '';
}
searchBar(false);
$('#btnGo').disabled = false; $('#btnLucky').disabled = false;
}

/* "Hôtel familial ou appart-hôtel" → "Hôtel familial" ; "180-250€ par nuit" → "1
const shortType = s => String(s || 'logement').split(/\s*(?:\(|\bou\b|\)/i)[0].t

```



```

const cleanPrix = s => String(s || '').replace(/\s*(par|\/)\s*nuit/gi, '').trim()

function renderDestinations(d) {
  const zone = $('#zoneResults');
  const n = (d.destinations || []).length;
  let html = `<div class="card"><h2>${n > 1 ? 'Compare tes ' + n + ' voyages' : '
  <p class="sub">${n > 1 ? 'Des propositions volontairement différentes. Compare-
  <div class="dest-grid">`;
  (d.destinations || []).forEach((x, i) => {
    const tIco = ({ avion: '✈️', train: '🚂', voiture: '🚗' })[x.transport_conseille] ||
    html += `<div class="dest" data-i="${i}">
      <div class="flag">${esc(x.drapeau || '📍')}</div>
      <h3>${esc(x.nom)}</h3><div class="country">${esc(x.pays)}</div>
      <p>${esc(x.resume)}</p>
      <div class="dest-facts">
        <div class="fact"><span class="fk">💰 Budget</span><span class="fv">${esc(
        <div class="fact"><span class="fk">${tIco} ${esc(x.transport_conseille || '
        <div class="fact"><span class="fk">🏠 ${esc(shortType(x.logement_type))}<
        <div class="fact"><span class="fk">☀️ Météo</span><span class="fv">${esc(
        <div class="fact"><span class="fk">🕒 Durée</span><span class="fv">${esc(
        <div class="fact"><span class="fk">🗣️ Langue</span><span class="fv">${esc(
        </div>
        ${x.transport_pourquoi || x.logement_pourquoi} ? `<p class="hint" style="m
        <div class="tags" style="margin-top:10px">${(x.points_forts || []).map(p => `<s
        <button class="btn sm" style="width:100%;justify-content:center;margin-top:
        </div>`;
    });
  html += `</div>`;
  /* Les questions ne s'affichent QUE dans la pop-up – jamais dans la page. */
  html += `</div>`;
  zone.innerHTML = html;

  $('<div class="dest">').forEach(el => el.onclick = () => chooseTrip(+el.dataset.i));
  $('<div class="chip">').forEach(el => el.onclick = () => {
    state.propAnswers = state.propAnswers || [];
    state.propAnswers.push(`${el.dataset.q || 'Affinage'} → ${el.dataset.r}`.slice(0, 12));
    state.propAnswers = state.propAnswers.slice(-12);
    save();
    toast('✓ ' + el.dataset.r);
    proposeTrips(state.propAnswers.join(' · '));
  });
}

const LS_HIST = 'acolate_history';
function getHistory() { try { return JSON.parse(localStorage.getItem(LS_HIST)) || [] } catch {} }
function pushHistory(t) {
  const h = getHistory().filter(x => x.nom !== t.nom);

```

```

    h.push({ nom: t.nom, pays: t.pays, quand: Date.now() });
    localStorage.setItem(LS_HIST, JSON.stringify(h.slice(-10)));
}

function chooseTrip(i){
    state.trip = state.destinations[i];
    pushHistory(state.trip);
    state.cache = {}; state.checklist = {}; state.spends = []; state.chatLog = [];
    state._geo = null; state.planAnswers = []; state._qsDone = false; _onSiteDone =
    save();
    unlockSteps();
    toast(`Cap sur ${state.trip.nom} ! ✈`);
    gotoStep(3);
}

/* =====
BOARDING PASS
===== */
function passHTML(){
    const t = state.trip, p = state.prefs || {};
    if(!t) return '';
    const from = (p.from || 'PAR').slice(0,3).toUpperCase();
    const to   = (t.iata || t.nom.slice(0,3)).toUpperCase();
    const plan = state.cache.plan;
    const d     = stayDates();
    const jj    = s => s.split('-').reverse().slice(0,2).join('/');
    const dates = d ? `${jj(d.in)} → ${jj(d.out)}` : (p.when || 'dates flexibles');
    const nuits = d ? Math.max(1, Math.round((new Date(d.out) - new Date(d.in)) / 8
    const pax    = `${p.adults || 1} ad.${p.kids ? ` + ${p.kids} enf.` : ''}`;
    /* on garde un type de logement COURT : "Appartement ou Hôtel familial" → "Appa
    const logt = plan?.logement
        ? String(plan.logement.type || '').split(/\s*(?:\(|ou |\|\/)/)[0].trim().slice(
        : '';
    const budget = plan?.budget?.total ? `${plan.budget.total} €` : (t.budget_estim

return `<div class="pass">
    <div class="pass-top">
        <div class="pass-route">
            <span class="iata">${esc(from)}</span>
            <span class="dash"></span>
            <span class="plane">✈</span>
            <span class="dash"></span>
            <span class="iata">${esc(to)}</span>
        </div>
        <button class="pass-change" data-changedest title="Changer de destination">
    </div>

```

```

<div class="pass-info">
  <div class="pi"><span class="pk">Destination</span><span class="pv">${esc(t
  <div class="pi"><span class="pk">Dates</span><span class="pv">${esc(dates)}
  <div class="pi"><span class="pk">Passagers</span><span class="pv">${esc(pax
  <div class="pi"><span class="pk">Budget</span><span class="pv">${esc(budget
</div>

<div class="pass-tear">
  <div class="barcode">${'<i></i>'.repeat(26)}</div>
  <div class="pass-acts">
    <button class="pact" data-passpng title="Télécharger le ticket avec son Q
    <button class="pact" data-sharelink title="Partager un lien qui importe c
    <button class="pact" data-ics title="Ajouter le programme à ton agenda">1
  </div>
</div>

<p class="pass-note">Ticket souvenir – ne permet pas d'embarquer. Le QR sert
</div>`;
}
function changeDest(){ gotoStep(1); }
window.changeDest = changeDest;
function refreshPasses(){
  const h = passHTML();
  ['#passSlot2', '#passSlot3', '#passSlot4', '#passSlot5'].forEach(s=>{ const el=$(s
}

/* =====
NAVIGATION
===== */
function unlockSteps(){
  $$('.step').forEach(s => {
    const n = +s.dataset.step;
    if(n === 2) s.classList.toggle('locked', !(state.destinations||[]).length);
    if(n === 3) s.classList.toggle('locked', !state.trip);
  });
}
let _onSiteDone = false;
function openSub(t){
  $$('.subtab').forEach(x => x.classList.toggle('on', x.dataset.t === t));
  Object.entries(TAB_PANELS).forEach(([k, sel]) => $(sel)?.classList.toggle('hidd
}
function gotoStep(n, sub){
  n = Math.min(n, 3);
  if(n === 2 && !(state.destinations||[]).length){ toast('Remplis d'abord le ques
  if(n === 3 && !state.trip){ toast('Choisis d'abord un des 3 voyages 🤔'); retur
  state.step = n; save();
  $$('.step').forEach(s => s.classList.toggle('active', +s.dataset.step === n));
  [1,2,3].forEach(i => $('#view'+i).classList.toggle('hidden', i !== n));

```

```

    refreshPasses();
    window.scrollTo({top:0, behavior:'smooth'});
    if(n === 3) loadPlan();
}
$$('.step').forEach(s => s.onclick = () => { if(!s.classList.contains('locked'))

/* =====
    ÉTAPE 2 – PLAN CLÉ EN MAIN (Gemini · heavy)
    L'IA choisit le transport, le logement, organise le séjour,
    pose ses questions – le voyageur n'a plus qu'à valider.
===== */
function syncModeFromPlan(d){
    const map = {avion:'plane', train:'train', voiture:'car'};
    const m = map[d?.transport?.mode];
    if(m && !state.modeManual) state.mode = m;
    save();
    $('#tgPlane').classList.toggle('on', state.mode==='plane');
    $('#tgCar').classList.toggle('on', state.mode==='car');
    $('#tgTrain').classList.toggle('on', state.mode==='train');
}

/* =====
    DONNÉES RÉELLES GRATUITES – ancrent l'IA dans le concret
    (Open-Meteo géocodage+météo, Wikipédia, prix de vols captés)
===== */
async function geoPlace(name){
    try{
        const r = await fetch(`https://geocoding-api.open-meteo.com/v1/search?name=${
        const d = await r.json();
        return d.results?.[0] || null;
    }catch(e){ return null; }
}
function havKm(a, b){
    const R = 6371, rad = x => x * Math.PI / 180;
    const dLa = rad(b.latitude - a.latitude), dLo = rad(b.longitude - a.longitude);
    const h = Math.sin(dLa/2)**2 + Math.cos(rad(a.latitude))*Math.cos(rad(b.latitude))
    return 2 * R * Math.asin(Math.sqrt(h));
}
const _avg = arr => Math.round(arr.reduce((x,y)=>x+y,0) / arr.length);

const COUNTRY_CC = { 'france':'FR','italie':'IT','espagne':'ES','portugal':'PT','

async function realData(){
    if(SET?.reels === false) return ''; /* le voyageur a désactivé les données ré
    const t = state.trip; if(!t) return '';
    const key = t.nom + ',' + t.pays;
    let R = state.cache._real;

```

```

if(!R || R.key !== key){
  R = { key };
  try{
    const [g1, g2] = await Promise.all([ geoPlace(state.prefs?.from || 'Paris')
    if(g1 && g2) R.dist = Math.round(havKm(g1, g2));
    if(g2){
      const depDate = state.prefs?.depart ? new Date(state.prefs.depart) : null
      const farAway = depDate && (depDate - Date.now()) > 16 * 86400000;
      if(farAway){
        /* départ lointain → climat réel du même mois l'an dernier (archive Ope
        const y = depDate.getFullYear() - 1, m = String(depDate.getMonth() + 1)
        const last = new Date(y, depDate.getMonth() + 1, 0).getDate();
        const wa = await fetch(`https://archive-api.open-meteo.com/v1/archive?l
        if(wa?.daily?.temperature_2m_max?.length){
          const rain = Math.round((wa.daily.precipitation_sum||[]).reduce((a,b)
          R.meteo = `climat typique du mois du voyage (relevés réels ${m}/${y})
          R.mNums = { min:_avg(wa.daily.temperature_2m_min), max:_avg(wa.daily.
        }
      }
      if(!R.meteo){
        const w = await fetch(`https://api.open-meteo.com/v1/forecast?latitude=
        if(w.daily?.temperature_2m_max?.length){
          R.meteo = `${_avg(w.daily.temperature_2m_min)}°C à ${_avg(w.daily.tem
          R.mNums = { min:_avg(w.daily.temperature_2m_min), max:_avg(w.daily.te
        }
      }
    }catch(e){}
    const _wikiP = fetch(`https://fr.wikipedia.org/api/rest_v1/page/summary/${enc
      .then(r => r.ok ? r.json() : null).then(wk => { if(wk?.extract) R.wiki = wk
    /* Wikivoyage : conseils orientés voyageur (quartiers, transports, arnaques)
    const _wvP = fetch(`https://fr.wikivoyage.org/api/rest_v1/page/summary/${enco
      .then(r => r.ok ? r.json() : null).then(wv => { if(wv?.extract) R.wv = wv.e
    /* Jours fériés officiels du pays pendant le séjour (Nager.Date, sans clé) */
    const _holP = (async () => { try{
      const dts = stayDates();
      if(!dts) return;
      const cc = COUNTRY_CC[String(t.pays||'').toLowerCase()];
      if(!cc) return;
      const yr = dts.in.slice(0, 4);
      const hs = await fetch(`https://date.nager.at/api/v3/PublicHolidays/${yr}/${
      if(!Array.isArray(hs)) return;
      const inRange = hs.filter(h => h.date >= dts.in && h.date <= dts.out)
        .map(h => `${h.date} (${h.localName})`);
      if(inRange.length) R.feries = inRange.join(', ');
    }catch(e){} })();
    /* horaires de train réels (Deutsche Bahn) — seulement si le rail est plausib

```

```

const _trainP = (async () => { try{
  if(!R.dist || R.dist < 1600){
    const tOut = new Promise(res => setTimeout(() => res(null), 8000));
    const sPair = Promise.all([ dbStation(state.prefs?.from || 'Paris'), dbSt
    const st = await Promise.race([sPair, tOut]);
    if(st && st[0] && st[1]){
      const rj = await Promise.race([
        fetch(`${DB_API}/journeys?from=${st[0].id}&to=${st[1].id}&results=3&t
        tOut
      ]);
      if(rj && rj.ok){
        const dj = await rj.json();
        const js = (dj.journeys||[]).filter(j => j.legs?.length);
        if(js.length){
          const best = js[0];
          const dep = new Date(best.legs[0].departure), arr = new Date(best.l
          const mins = Math.round((arr - dep) / 60000);
          const chg = Math.max(0, best.legs.filter(l => !l.walking).length -
          const prices = js.map(j => j.price?.amount).filter(Boolean);
          if(mins > 0) R.train = `${Math.floor(mins/60)}h${String(mins%60).pa
        }
      }
    }
  }
} catch(e){} })();
/* taux de change réel (Frankfurter, BCE) */
const _fxP = (async () => { try{
  const code = ((t.monnaie||'').toUpperCase().match(/\b(?:!EUR) [A-Z]{3}\b/))||[
  if(code){
    const rf = await fetch(`https://api.frankfurter.dev/v1/latest?base=EUR&sy
    if(rf.ok){
      const df = await rf.json();
      if(df.rates?.[code]) R.fx = `1 € = ${df.rates[code].toFixed(2)} ${code}
    }
  }
} catch(e){} })();
await Promise.allSettled([_wikiP, _wvP, _holP, _trainP, _fxP]);
state.cache._real = R; save();
}
const L = [];
if(R.dist) L.push(`Distance ${state.prefs?.from || 'départ'} → ${t.nom} : envir
if(R.meteo) L.push(`Météo réelle à ${t.nom} en ce moment : ${R.meteo}`);
if(state.cache.realPrice) L.push(`Prix de vol réel constaté par nos moteurs : $
if(R.train) L.push(`Trajet en TRAIN réel ${state.prefs?.from || 'départ'} → ${t
if(R.fx) L.push(`Taux de change réel : ${R.fx} — utilise CE taux pour toute con
if(R.wiki) L.push(`Contexte factuel (Wikipédia) : ${R.wiki}`);
if(R.wv) L.push(`Infos voyageur (Wikivoyage) : ${R.wv}`);

```

```

    if(R.feries) L.push(`JOURS FÉRIÉS OFFICIELS pendant le séjour : ${R.feries} - b
    return L.length ? `\\nDONNÉES RÉELLES VÉRIFIÉES - appuie-toi dessus, ne les cont
}

/* --- Relecture croisée : Groq vérifie le plan de Gemini, Gemini corrige si beso
async function reviewPlan(d, basePrompt){
    if(!hasGroq()) return d;
    try{
        const v = await groq(`Tu es un vérificateur impitoyable de plans de voyage. $
PLAN À VÉRIFIER (JSON) : ${JSON.stringify(d).slice(0, 5500)}
Contrôle STRICTEMENT et uniquement ces 6 points :
1. "budget.total" respecte-t-il le budget demandé par personne ?
2. Le nombre d'entrées de "programme" correspond-il à la durée demandée ?
3. Le transport choisi est-il cohérent avec la distance et les limites du voyageu
4. COHÉRENCE GÉOGRAPHIQUE : chaque journée regroupe-t-elle des lieux PROCHES les
5. Les lieux cités existent-ils VRAIMENT dans cette ville, et sont-ils ouverts à
6. Le programme est-il réaliste en temps (pas 6 musées dans une seule journée) ?
Réponds en JSON : {"ok":true} si tout est cohérent, sinon {"ok":false,"problemes"
    if(v?.ok !== false || !(v.problemes||[]).length){ d._checked = 'ok'; return d
    const d2 = await gemini(basePrompt + `\\n\\nATTENTION - une relecture indépenda
    d2._checked = 'fixed';
    return d2;
    }catch(e){ return d; }
}

async function loadPlan(force = false){
    const zone = $('#zonePlan');
    if(state.cache.plan && !force){ renderPlan(state.cache.plan); syncModeFromPlan(
    zone.innerHTML = loaderHTML('Acolite organise ton voyage de A à Z...');
    const t = state.trip;
    const answers = (state.planAnswers||[]).join(' · ');
    const realCtx = await realData();
    /* Le transport et le logement ont DÉJÀ été trouvés à l'étape 2 : on les garde
    const dejaTrouve = (t.transport_conseille || t.logement_quartier)
        ? `\\nCHOIX DÉJÀ VALIDÉS À L'ÉTAPE 2 (le voyageur les a acceptés en choisissan
- Transport : ${t.transport_conseille || '?'}${t.transport_prix ? ` (${t.transpor
- Logement : ${t.logement_type || '?'} dans le quartier ${t.logement_quartier ||
TON TRAVAIL : approfondir (détails pratiques, programme jour par jour, budget pré
        : '';
    const prompt = `Tu es Acolite, organisateur de voyage expert. ${ctx()}
    ${realCtx}${dejaTrouve}
    Destination validée : ${t.nom} (${t.pays}).
    RÈGLE ABSOLUE : ne cite que des quartiers, lieux et établissements RÉELS et vérif
    Si les données réelles incluent un trajet en train ou un taux de change, appuie t
    ${answers ? 'RÉPONSES du voyageur à tes questions précédentes (à intégrer au plan
    MISSION : organise TOUT le voyage. C'est TOI qui décides du transport (avion, tra

```

INTERDIT de poser des questions : renvoie "questions":[]. Le voyageur a déjà tout

Réponds UNIQUEMENT en JSON. Commence OBLIGATOIREMENT par le champ "analyse" (rais

```
{
  "analyse":"3-4 phrases : comparaison chiffrée des transports possibles (durée/pr
  "transport":{
    "mode":"avion" ou "train" ou "voiture",
    "pourquoi":"2 phrases : pourquoi CE transport vu le budget et les conditions",
    "details":"trajet concret : aéroports/gares/axes, durée, ce qu'il faut réserver
    "prix_estime":"fourchette réaliste A/R par personne"
  },
  "logement":{
    "type":"1 ou 2 mots MAXIMUM : hôtel, appartement, auberge, villa...",
    "quartier":"quartier précis recommandé",
    "prix_nuit":"fourchette en € uniquement, ex 80-120€ (sans le mot nuit)",
    "pourquoi":"1 phrase"
  },
  "programme":[{"jour":1,"resume":"le thème du jour en 1 ligne","lieux":["2-4 lieu
  "budget":{"total":nombre entier en euros par personne,"repartition":"1 phrase :
  "conseil_cle":"LE conseil le plus important pour ce voyage",
  "questions":[{"texte":"question courte","options":["3-4 réponses courtes"]}]}
}
```

Le programme couvre toute la durée (\${state.prefs.days}), 1 ligne par jour.`;

```
try{
  const tok = { court: 4096, normal: 8192, long: 12288 }[SET?.detail || 'normal
  let d = await gemini(prompt, true, tok, false, 0.45);
  if(SET?.verif !== false) d = await reviewPlan(d, prompt); /* relecture croi
  state.cache.plan = d; save();
  renderPlan(d);
  syncModeFromPlan(d);
}catch(e){
  if(e.message!=='NO_KEY') zone.innerHTML = errHTML('Organisation impossible po
}
}
```

```
/* =====
HOTELLOOK (Travelpayouts) - vrais prix d'hôtels dans l'app
Endpoint cache.json : prix agrégés Booking/Expedia/Agoda.
Repli automatique sur les liens comparateurs si indisponible.
===== */
```

```
/* Relais CORS : Hotellook n'envoie pas les en-têtes CORS, le navigateur
refuse donc de lire sa réponse. On passe par un relais qui, lui, les ajoute.
Ordre : ton Worker Cloudflare (config.js) → relais publics → appel direct. */
function relays(){
  const own = API();
  const list = [];
  if(own) list.push({ nom:'ton backend', wrap: u => `${own}?url=${encodeURIComponent
```



```

    list.push({ nom:'relais public 1', wrap: u => `https://corsproxy.io/?url=${enco
    list.push({ nom:'relais public 2', wrap: u => `https://api.allorigins.win/raw?u
    list.push({ nom:'direct', wrap: u => u });
    return list;
  }

```

```

async function loadHotels(force = false){
  const zone = $('#zoneHotels');
  if(!zone) return;
  const t = state.trip;
  const d = stayDates();
  const token = tpKey();
  if(!t || !d || (!token && !useBackend())){
    zone.innerHTML = `
```

```

try{
  const res = await fetch(r.wrap(api), { headers: { 'Accept':'application/json' } });
  if(!res.ok) throw new Error('HTTP ' + res.status);
  const txt = await res.text();
  let data;
  try{ data = JSON.parse(txt); }catch(e){ throw new Error('réponse illisible'
  const rows = (Array.isArray(data) ? data : []).filter(h => h.priceFrom || h
  if(!rows.length) throw new Error('aucun tarif pour ces dates');
  rows.sort((a, b) => (a.priceFrom || a.priceAvg) - (b.priceFrom || b.priceAv
  state.cache[ck] = rows.slice(0, 8); save();
  localStorage.setItem('acolite_relay', r.nom);
  renderHotels(state.cache[ck], r.nom);
  return;
}catch(e){
  const cors = e instanceof TypeError || e.name === 'TypeError' || /failed to
  errs.push(`${r.nom} : ${cors ? 'bloqué par le navigateur (CORS)' : e.messag
  }
}
localStorage.removeItem('acolite_relay');
zone.innerHTML = `
```

```

    }).join('') + `<p class="hint">Prix réels agrégés (Booking, Expedia, Agoda...) po
}

document.addEventListener('click', e => {
  if(e.target.id === 'hotRetry'){ e.preventDefault(); loadHotels(true); }
});

/* --- Liens logement pré-remplis (comparateurs + sites directs) --- */
function stayDates(){
  const p = state.prefs || {};
  if(!p.depart) return null;
  let days = 7;
  const m = String(p.days||'').match(/\d+/g);
  if(/semaine/i.test(p.days||'')) days = (m ? +m[m.length-1] : 1) * 7;
  else if(m) days = +m[m.length-1];
  days = Math.min(30, Math.max(2, days));
  return { in: p.depart, out: addDays(p.depart, days) };
}

function stayLinks(place){
  const p = state.prefs || {}, t = state.trip || {};
  const q = `${place ? place + ', ' : ''}${t.nom || ''}`;
  const d = stayDates();
  const A = p.adults || 2, K = p.kids || 0;
  const enc = encodeURIComponent;
  return {
    cozy: `https://www.cozycozy.com/fr/s/${enc(((t.nom||'') + (t.pays ? '--' +
    hometogo: `https://www.hometogo.fr/search/?q=${enc(q)}${d ? `&arrival=${d.in}&
    booking: `https://www.booking.com/searchresults.fr.html?ss=${enc(q)}${d ? `&c
    airbnb: `https://www.airbnb.fr/s/${enc(q)}/homes?adults=${A}${K ? `&children
    abritel: `https://www.abritel.fr/search?destination=${enc(q)}${d ? `&startDat
  };
}

function renderPlan(d){
  const icons = {avion:'✈️', train:'🚆', voiture:'🚗'};
  const tr = d.transport||{}, lg = d.logement||{}, bd = d.budget||{};
  const qs = (d.questions||[]).filter(q=>q && q.texte);
  const A = (state.prefs?.adults||1) + (state.prefs?.kids||0);
  const dts = stayDates();
  const nuits = dts ? Math.max(1, Math.round((new Date(dts.out) - new Date(dts.in

let html = `
  <p class="sub" style="margin:-4px 0 12px">Le voyage qu'Acolite a construit po

  <h3 style="margin:0 0 10px">📦 L'essentiel</h3>
  <div class="plan-grid">
    <div class="plan-stat"><div class="k">Comment y aller</div><div class="v">$

```

```

    <div class="plan-stat"><div class="k">Où dormir</div><div class="v">🏠 ${es
    <div class="plan-stat"><div class="k">Ce que ça coûte</div><div class="v">${
    ${state.cache._real?.mNums ? `<div class="plan-stat wx"><canvas id="wxCv" w
    </div>

<div class="divider"></div>
<h3 style="margin:0 0 10px">🧠 Pourquoi ces choix</h3>
<div class="item" style="align-items:flex-start"><div class="emo">${icons[tr.
    <div style="flex:1"><h4>Y aller en ${esc(tr.mode||'transport')}</h4><p>${es
    ${tr.details ? `<p class="hint" style="margin-top:4px">${esc(tr.details)}</
    <div class="item" style="align-items:flex-start"><div class="emo">🏠</div>
    <div style="flex:1"><h4>Dormir à ${esc(lg.quartier||'-')}</h4><p>${esc(lg.p
    ${bd.repartition ? `<div class="item" style="align-items:flex-start"><div cla
    <div style="flex:1"><h4>Le budget en détail</h4><p>${esc(bd.repartition)}</
    ${d.conseil_cle ? `<div class="item" style="align-items:flex-start;background
    <div style="flex:1"><h4 style="color:#101010">Le conseil à retenir</h4><p s

<div class="divider"></div>
<h3 style="margin:0 0 4px">📅 Ton programme, jour par jour</h3>
<p class="hint" style="margin:0 0 12px">Une journée ne te convient pas ? Le b
    ${ (d.programme||[]).map(jr=>`
        <div class="item" style="align-items:flex-start">
            <span class="tl-time" style="flex-shrink:0">J${esc(String(jr.jour))}</spa
            <div style="flex:1;min-width:0">
                <h4>${esc(jr.resume)}</h4>
                ${ (jr.lieux||[]).length ? `<p class="hint" style="margin:3px 0 0">📍 ${
            </div>
            <button class="btn sm ghost" data-planb="${esc(String(jr.jour))}" title="
    </div>`).join('')}`;

if(qs.length){
    html += `<div class="divider"></div><h3 style="margin:0 0 4px">😓 Affine enc
        <p class="hint" style="margin:0 0 10px">Réponds et Acolite adapte le voyage
    qs.forEach(q=>{
        html += `<h4 style="margin:10px 0 6px;font-family:'Sora'">${esc(q.texte)}</
        <div class="chips">${ (q.options||[]).map(o=>`<div class="chip planq" data-q
    });
}

html += `
    <div class="divider"></div>
    <h3 style="margin:0 0 4px">👉 Et maintenant ?</h3>
    <p class="hint" style="margin:0 0 4px"><strong>Réservation</strong> : tous le
    <strong>Simulation</strong> : les vrais prix des vols et des trains, jour par
    <div class="row" style="margin-top:14px">
        <button class="btn sm ghost" id="btnPlanRedo">🔄 Tout réorganiser</button>
        <button class="btn sm ghost" data-changedest>📍 Changer de destination</butt

```

```

</div>`;

$('#zonePlan').innerHTML = html;
fitStats();
refreshPasses();
startWx();
}
addEventListener('resize', () => { if($('#zonePlan')?.querySelector('.plan-stat')

document.addEventListener('click', e => {
  if(e.target.id === 'btnJournal'){
    const txt = $('#jrText').value.trim();
    if(!txt && !_jrPhotos.length){ toast('Écris quelque chose ou ajoute une photo
    const j = getJournal();
    const k = _jrKey();
    j[k] = j[k] || [];
    j[k].push({ jour: +$('#jrDay').value || 1, texte: txt.slice(0, 1200), photos:
    if(saveJournal(j)){
      renderJournal();
      toast('📅 Souvenir enregistré');
    }
    return;
  }
  const del = e.target.closest('[data-jr]');
  if(del){
    if(!confirm('Supprimer ce souvenir ?')) return;
    const j = getJournal(), k = _jrKey();
    const sorted = (j[k]||[]).slice().sort((a,b) => a.jour - b.jour || a.ts - b.t
    const victim = sorted[+del.dataset.jr];
    j[k] = (j[k]||[]).filter(x => x.ts !== victim.ts);
    saveJournal(j); renderJournal();
  }
});

/* =====
  ALERTE PRIX — mémorise le prix du vol et signale les baisses
===== */
const LS_PRICES = 'acolite_prices';
function trackPrice(prix, source){
  if(!state.trip || !prix) return;
  let all;
  try{ all = JSON.parse(localStorage.getItem(LS_PRICES)) || {}; }catch(e){ all =
  const k = `${state.trip.nom}_${state.prefs?.depart || 'flex'}`;
  const hist = all[k] || [];
  const last = hist[hist.length - 1];
  /* on n'enregistre qu'un point par jour et par source */

```

```

const today = new Date().toISOString().slice(0, 10);
if(!last || last.d !== today || last.p !== prix){
  hist.push({ p: prix, d: today, s: source });
  all[k] = hist.slice(-30);
  try{ localStorage.setItem(LS_PRICES, JSON.stringify(all)); }catch(e){}
}
/* comparaison avec le meilleur prix connu (hors aujourd'hui) */
const anciens = hist.filter(h => h.d !== today);
if(!anciens.length) return;
const ref = Math.min(...anciens.map(h => h.p));
const diff = prix - ref;
const bar = $('#priceAlert');
if(!bar) return;
if(diff <= -5){
  bar.style.display = 'block';
  bar.className = 'item';
  bar.innerHTML = `<div class="emo">📉</div><p style="flex:1;font-weight:800">E
} else if(diff >= 15){
  bar.style.display = 'block';
  bar.className = 'item';
  bar.innerHTML = `<div class="emo">📈</div><p style="flex:1;font-weight:800">I
} else {
  bar.style.display = 'none';
}
}

```

```

/* --- PLAN B : régénère UNE seule journée (sans tout refaire) --- */
async function planB(jour){
  const d = state.cache.plan;
  if(!d?.programme) return;
  const jr = d.programme.find(x => +x.jour === +jour);
  if(!jr) return;
  const raison = prompt(`Pourquoi refaire le jour ${jour} ?\n(ex : il pleut, on e
if(raison === null) return;
toast('🔄 Nouvelle version du jour ' + jour + '...');
try{
  const autres = d.programme.filter(x => +x.jour !== +jour).map(x => `J${x.jour
  const R = state.cache._real || {};
  const nd = await gemini(`Tu réorganises UNE SEULE journée d'un voyage à ${sta

JOUR À REFAIRE : jour ${jour} - actuellement "${jr.resume}" (${(jr.lieux||[]).joi
RAISON DU CHANGEMENT : "${String(raison).slice(0,160)}"
${R.meteo ? `MÉTÉO RÉELLE : ${R.meteo}` : ''}
${R.feries ? `JOURS FÉRIÉS : ${R.feries}` : ''}

```

AUTRES JOURNÉES (ne les répète PAS, ne propose PAS les mêmes lieux) :

```
 ${autres || 'aucune'}
```

Propose une nouvelle journée qui répond à la raison donnée (s'il pleut → activité Uniquement des lieux qui EXISTENT VRAIMENT, regroupés dans le même quartier.

Réponds UNIQUEMENT en JSON : {"jour":\${jour},"resume":"phrase courte","lieux":["L

```
    if(!nd?.resume) throw new Error('vide');
    d.programme = d.programme.map(x => +x.jour === +jour ? { jour:+jour, resume:n
    state.cache.plan = d; save();
    renderPlan(d);
    toast('✅ Jour ' + jour + ' réorganisé');
  }catch(e){ toast('❌ Impossible de refaire cette journée'); }
}
document.addEventListener('click', e => {
  const b = e.target.closest('[data-planb]');
  if(b) planB(b.dataset.planb);
});
```

```
/* --- Accordéons + boutons "changer de destination" (CSP stricte : aucun onclick
document.addEventListener('click', e => {
  const acc = e.target.closest('[data-acc]');
  if(acc){ acc.parentElement.classList.toggle('open'); return; }
  if(e.target.closest('[data-changedest]')) changeDest();
});
```

```
/* --- Pop-up Questions : réponses obligatoires avant le voyage final --- */
let _qsList = [];
function openQsPopup(qs){
  _qsList = qs.slice(0, 3);
  const pg = $('#qsProg');
  if(pg) pg.innerHTML = _qsList.map(() => '<i></i>').join('');
  $('#zoneQs').innerHTML = _qsList.map((q, i) => `
    <h4 style="margin:14px 0 6px;font-family:'Sora'">${i+1}. ${esc(q.texte)}</h4>
    <div class="chips" data-qi="${i}">${(q.options||[]).map(o=>`<div class="chip
  $('#btnQsGo').disabled = true;
  $('#ovQs').classList.add('show');
}
document.addEventListener('click', e => {
  const c = e.target.closest('.chip.qsopt');
  if(c){
    $$(`.chip.qsopt[data-qi="${c.dataset.qi}"]`).forEach(x => x.classList.remove(
    c.classList.add('on');
    const reste = $$('#zoneQs .chips').filter(g => !g.querySelector('.on')).length
    $('#btnQsGo').disabled = reste > 0;
    $$('#qsProg i').forEach((el, i) => el.classList.toggle('on', !!$$('#zoneQs .c
    const btn = $('#btnQsGo');
    if(btn) btn.textContent = reste ? `Encore ${reste} question${reste > 1 ? 's'`
```

```

    return;
}
if(e.target.id === 'btnQsGo'){
    $$('#zoneQs .chip.qsopt.on').forEach(c2 => {
        state.propAnswers = state.propAnswers || [];
        state.propAnswers.push(`${_qsList[+c2.dataset.qi]?.texte} → ${c2.dataset.a}
    });
    state.propAnswers = (state.propAnswers || []).slice(-12);
    state._qsDone = true; save();
    $('#ovQs').classList.remove('show');
    toast('🎉 Merci – Acolite affine tes propositions...');
    proposeTrips(state.propAnswers.join(' · ')); /* on relance les PROPOSITIONS
    return;
}
if(e.target.id === 'btnQsSkip'){
    state._qsDone = true; save();
    $('#ovQs').classList.remove('show');
    toast('Ok, Acolite garde ses propositions actuelles 👍');
}
});

/* --- Pop-up Réservation : tous les liens + prix réels d'hôtels --- */
function buildResa(){
    const t = state.trip || {}, p = state.prefs || {};
    const enc = encodeURIComponent;
    const L = stayLinks(state.cache.plan?.logement?.quartier || '');
    const d = stayDates();
    const q = `${p.from||'Paris'} ${t.nom||''}`;
    const gf = `https://www.google.com/travel/flights?q=${enc('vols ' + q + (d ? '
    const sky = `https://www.skyscanner.fr/`;
    const rya = `https://www.ryanair.com/fr/fr`;
    const sncf = `https://www.sncf-connect.com/`;
    const trl = `https://www.thetrainline.com/fr`;
    const omio = `https://www.omio.fr/`;
    const car = `https://www.google.com/maps/dir/${enc(p.from||'Paris')}/${enc((t.n
    const gyg = `https://www.getyourguide.fr/s/?q=${enc(t.nom||'')}`;
    const cvt = `https://www.civitatis.com/fr/recherche/?q=${enc(t.nom||'')}`;
    const ta = `https://www.tripadvisor.fr/Search?q=${enc(t.nom||'')}`;
    const B = (href, label, solid) => `<a id="zoneHotels"></a>

```



```

<h3 style="margin:14px 0 8px">🔍 Comparer tous les logements</h3>
<div class="row">${B(L.cozy, 'Cozycozy - comparateur', 1)}${B(L.hometogo, 'HomeT
<div class="row" style="margin-top:8px">${B(L.booking, 'Booking')}${B(L.airbnb
<p class="hint">Recherches pré-remplies : ${esc(state.cache.plan?.logement?.q
<div class="divider"></div>
<h3 style="margin:0 0 8px">🎡 Activités & visites</h3>
<div class="row">${B(gyg, 'GetYourGuide', 1)}${B(cvt, 'Civitatis')}${B(ta, 'Tripa
}

```

```

const _e1 = $('#btnOpenResa'); if(_e1) _e1.onclick = () => {
  if(!state.trip){ toast('Choisis d'abord un voyage 🤔'); return; }
  buildResa();
  $('#ovResa').classList.add('show');
  loadHotels();
};
const _e2 = $('#btnOpenSim'); if(_e2) _e2.onclick = () => {
  if(!state.trip){ toast('Choisis d'abord un voyage 🤔'); return; }
  $('#ovSim').classList.add('show');
  loadTransport();
};
document.addEventListener('click', e => {
  const c = e.target.closest('[data-close]');
  if(c){ $('##' + c.dataset.close).classList.remove('show'); return; }
  if(e.target.classList?.contains('overlay')) e.target.classList.remove('show');
});

```

```

function planValidate(){
  const d = state.cache.plan;
  if(!d){ toast("Le plan n'est pas encore prêt"); return; }
  const map = {avion:'plane', train:'train', voiture:'car'};
  state.mode = map[d.transport?.mode] || 'plane';
  state.modeManual = false;
  state.planOk = true;
  delete state.cache['transport_' + state.mode];
  save();
  loadTransport();
  toast(`Plan validé - billets ${d.transport?.mode||''} juste en dessous 📄`);
  $('#zoneTransport').scrollIntoView({behavior:'smooth', block:'start'});
}

```

```

/* Délégation : boutons du plan (la zone est re-rendue) */
document.addEventListener('click', e => {
  if(e.target.id === 'btnPlanOk'){ planValidate(); return; }
  if(e.target.id === 'btnPlanRedo'){ loadPlan(true); return; }
  const q = e.target.closest('.chip.planq');
  if(q){
    state.planAnswers = state.planAnswers || [];

```

```

    state.planAnswers.push(`${q.dataset.q} → ${q.dataset.a}`.slice(0,200));
    state.planAnswers = state.planAnswers.slice(-12);
    save();
    toast('Réponse prise en compte ✓');
    loadPlan(true);
  }
});

/* =====
ÉTAPE 3 – Y ALLER (Gemini · heavy)
===== */
const _e3 = $('#tgPlane'); if(_e3) _e3.onclick = () => setMode('plane');
const _e4 = $('#tgCar'); if(_e4) _e4.onclick = () => setMode('car');
const _e5 = $('#tgTrain'); if(_e5) _e5.onclick = () => setMode('train');
function setMode(m) {
  state.mode = m; state.modeManual = true; save();
  $('#tgPlane').classList.toggle('on', m==='plane');
  $('#tgCar').classList.toggle('on', m==='car');
  $('#tgTrain').classList.toggle('on', m==='train');
  loadTransport();
}

async function loadTransport(){
  const zone = $('#zoneTransport');
  const t = state.trip, p = state.prefs;
  const key = 'transport_' + state.mode;
  $('#tgPlane').classList.toggle('on', state.mode==='plane');
  $('#tgCar').classList.toggle('on', state.mode==='car');
  $('#tgTrain').classList.toggle('on', state.mode==='train');
  if(state.cache[key]){ renderTransport(state.cache[key]); return; }
  const msgs = {plane:'Analyse des vols...', car:'Calcul de la route...', train:'Rech
zone.innerHTML = loaderHTML(msgs[state.mode]);

  let prompt;
  if(state.mode === 'plane'){
    prompt = `Tu es Acolite, expert voyage. ${ctx()}
Le voyageur part en AVION de ${p.from} vers ${t.nom} (${t.pays}).
Réponds UNIQUEMENT en JSON :
{
  "aeroport_depart": "nom + code IATA le plus pratique depuis ${p.from}",
  "iata_depart": "code IATA seul, ex CDG",
  "aeroport_arrivee": "nom + code IATA",
  "iata_arrivee": "code IATA seul",
  "duree_vol": "ex: 2h15 direct",
  "prix_estime": "fourchette A/R réaliste pour cette période",
  "compagnies": ["3-4 compagnies pertinentes sur cette ligne"],
  "conseils": ["4 conseils concrets : quand réserver, quel jour partir moins cher,
```

```

}`;
    } else if(state.mode === 'car'){
        prompt = `Tu es Acolite, expert voyage. ${ctx()}`
        Le voyageur part en VOITURE de ${p.from} vers ${t.nom} (${t.pays}).
        Réponds UNIQUEMENT en JSON :
    {
        "distance":"ex: 950 km",
        "duree":"ex: 8h30 sans pause",
        "cout_estime":"carburant + péages, fourchette réaliste",
        "itineraire_resume":"axes principaux, ex: A6 puis A7...",
        "pauses":["2-3 super étapes sur la route (ville + pourquoi s'y arrêter)"],
        "conseils":["4 conseils concrets : vignettes/péages du pays, meilleure heure de
    }`;
    } else {
        prompt = `Tu es Acolite, expert voyage. ${ctx()}`
        Le voyageur part en TRAIN de ${p.from} vers ${t.nom} (${t.pays}).
        Réponds UNIQUEMENT en JSON :
    {
        "faisable":"oui" ou "non" ou "compliqué",
        "trajet":"description du trajet type : gares, correspondances, ex: Paris Gare de
        "duree":"durée totale estimée",
        "prix_estime":"fourchette réaliste A/R",
        "compagnies":["compagnies ferroviaires concernées"],
        "conseils":["4 conseils : quand réserver, pass éventuels (Interrail...), trains de
    }`;
    }
}

try{
    const d = await gemini(prompt);
    state.cache[key] = d; save();
    renderTransport(d);
}catch(e){
    if(e.message !== 'NO_KEY') zone.innerHTML = errHTML('Analyse impossible pour le
}

}

function renderTransport(d){
    const zone = $('#zoneTransport');
    const t = state.trip, p = state.prefs;
    if(state.mode === 'plane'){
        const from = d.iata_depart || 'PAR', to = d.iata_arrivee || (t.iata||'');
        const gf = `https://www.google.com/travel/flights?q=${encodeURIComponent(`vo
        const ky = `https://www.kayak.fr/flights/${encodeURIComponent(from)}-${encod
        const sky = `https://www.skyscanner.fr/transport/vols/${encodeURIComponent(fr
        zone.innerHTML = `
        <div class="grid" style="margin-bottom:14px">
            <div class="item"><div class="emo"><img alt="plane icon" data-bbox="530 930 550 945"/></div><div><h4>Départ</h4><p>${esc(c

```

```

    <div class="item"><div class="emo"><img alt="plane icon" data-bbox="530 48 555 63"/></div><div><h4>Arrivée</h4><p>${esc(
    <div class="item"><div class="emo"><img alt="clock icon" data-bbox="530 65 555 80"/></div><div><h4>Durée</h4><p>${esc(d.
    <div class="item"><div class="emo"><img alt="tag icon" data-bbox="530 82 555 97"/></div><div><h4>Prix estimé A/R</h4><
  </div>
  <div class="divider"></div>
  <h3 style="margin-bottom:10px">Chercher les billets <img alt="search icon" data-bbox="670 145 695 160"/></h3>
  <div class="row">
    <a class="btn" href="${esc(gf)}" target="_blank" rel="noopener">Google Fl
    <a class="btn ghost" href="${esc(ky)}" target="_blank" rel="noopener">Kay
    <a class="btn ghost" href="${esc(sky)}" target="_blank" rel="noopener">Sk
  </div>
  <p class="hint">Les liens ouvrent la recherche pré-remplie – compare les pr
  <div class="divider"></div>
  <h3 style="margin-bottom:6px"><img alt="tag icon" data-bbox="465 295 490 310"/> Prix réels en direct <span class="tag cyan
  <p class="hint" style="margin:0 0 12px">Deux moteurs : <strong>Ryanair</str
  <div class="grid tight" style="margin-bottom:12px">
    <div class="field"><label>Départ (IATA)</label><input id="ryFrom" maxleng
    <div class="field"><label>Arrivée (IATA)</label><input id="ryTo" maxlengt
    <div class="field"><label>Date aller</label><input id="ryDate" type="date
    <div class="field"><label>Flexibilité</label>
      <select id="ryFlex"><option value="3">± 3 jours</option><option value="
    </div>
    <div class="field"><label>Durée sur place</label>
      <select id="ryStay"><option value="3">2-3 nuits</option><option value="
    </div>
  </div>
  <div class="row">
    <button class="btn sm" id="btnRyRT"><img alt="magnifying glass icon" data-bbox="540 565 565 580"/> A/R Ryanair</button>
    <button class="btn sm ghost" id="btnRyCal"><img alt="calendar icon" data-bbox="605 582 630 597"/> Calendrier Ryanair</button>
    <button class="btn sm violet" id="btnTpAll"><img alt="globe icon" data-bbox="615 600 640 615"/> Toutes compagnies</button>
  </div>
  <div id="zoneRy" style="margin-top:14px"></div>`;
} else if(state.mode === 'car'){
  const saddr = encodeURIComponent(p.from);
  const daddr = encodeURIComponent(`${t.nom}, ${t.pays}`);
  const embed = `https://maps.google.com/maps?saddr=${saddr}&daddr=${daddr}&hl=
  const open = `https://www.google.com/maps/dir/${saddr}/${daddr}`;
  zone.innerHTML = `
    <div class="grid" style="margin-bottom:14px">
      <div class="item"><div class="emo"><img alt="ruler icon" data-bbox="530 795 555 810"/></div><div><h4>Distance</h4><p>${esc
      <div class="item"><div class="emo"><img alt="clock icon" data-bbox="530 812 555 827"/></div><div><h4>Durée</h4><p>${esc(d.
      <div class="item"><div class="emo"><img alt="tag icon" data-bbox="530 829 555 844"/></div><div><h4>Coût estimé</h4><p>${
      <div class="item"><div class="emo"><img alt="map icon" data-bbox="530 846 555 861"/></div><div><h4>Itinéraire</h4><p>${e
    </div>
    <div class="map-box" style="margin-bottom:16px"><iframe src="${esc(embed)}"
    <div class="row" style="margin-bottom:16px"><a class="btn" href="${esc(open
  `;

```

```

} else {
  const trl = `https://www.thetrainline.com/fr`;
  const sncf = `https://www.sncf-connect.com/`;
  const omio = `https://www.omio.fr/`;
  const fais = {oui: '✅ Faisable', 'non': '❌ Peu adapté', 'compliqué': '⚠️ Comp.
zone.innerHTML = `
  <div class="grid" style="margin-bottom:14px">
    <div class="item"><div class="emo">🚦</div><div><h4>Verdict</h4><p>${esc(
    <div class="item"><div class="emo">🕒</div><div><h4>Durée</h4><p>${esc(d.
    <div class="item"><div class="emo">💰</div><div><h4>Prix estimé A/R</h4><
    <div class="item"><div class="emo">🚆</div><div><h4>Trajet</h4><p>${esc(c
  </div>
  <div class="divider"></div>
  <h3 style="margin-bottom:10px">Chercher les billets 🎫</h3>
  <div class="row">
    <a class="btn" href="${trl}" target="_blank" rel="noopener">Trainline</a>
    <a class="btn ghost" href="${sncf}" target="_blank" rel="noopener">SNCF C
    <a class="btn ghost" href="${omio}" target="_blank" rel="noopener">Omio</
  </div>
  <div class="divider"></div>
  <h3 style="margin-bottom:6px">🕒 Horaires réels en direct <span class="tag
  <p class="hint" style="margin:0 0 12px">Vrais horaires (et parfois prix) in
  <div class="grid tight" style="margin-bottom:12px">
    <div class="field"><label>Gare / ville de départ</label><input id="dbFrom
    <div class="field"><label>Gare / ville d'arrivée</label><input id="dbTo"
    <div class="field"><label>Départ le</label><input id="dbWhen" type="datet
  </div>
  <button class="btn sm" id="btnDb">🔍 Chercher les trains</button>
  <div id="zoneDb" style="margin-top:14px"></div>`;
}
}

/* =====
  BILLETS EN DIRECT – APIs publiques gratuites sans clé
  ✈️ Ryanair farfnd (prix réels) · 🚆 v6.db.transport.rest (horaires DB)
  ===== */
function ryDefaultDate() {
  const base = state.prefs?.depart ? new Date(state.prefs.depart) : new Date(Date
  return base.toISOString().slice(0,10);
}
const addDays = (iso, n) => { const d = new Date(iso); d.setDate(d.getDate()+n);
const frDate = iso => new Date(iso).toLocaleDateString('fr-FR',{weekday:'short',d
const frTime = iso => new Date(iso).toLocaleTimeString('fr-FR',{hour:'2-digit',mi

/* --- ✈️ Ryanair : meilleurs A/R sur une fenêtre de dates --- */
async function ryRoundTrip() {
  const zone = $('#zoneRy');

```

```

const from = $('#ryFrom').value.trim().toUpperCase().replace(/[^A-Z]/g, '').slice(0, 3);
const to = $('#ryTo').value.trim().toUpperCase().replace(/[^A-Z]/g, '').slice(0, 3);
const date = $('#ryDate').value;
const flex = +$('#ryFlex').value, stay = +$('#ryStay').value;
if(from.length!==3 || to.length!==3 || !date){ toast('Renseigne 2 codes IATA + zone.innerHTML = loaderHTML('Interrogation des tarifs Ryanair...');
const outFrom = addDays(date, -Math.min(flex, Math.floor((new Date(date)-Date.now())/86400000));
const outTo = addDays(date, flex);
const inFrom = addDays(date, Math.max(1, stay - 2));
const inTo = addDays(date, stay + flex);
const url = `https://services-api.ryanair.com/farfnd/v4/roundTripFares?departur
+ `&outboundDepartureDateFrom=${outFrom}&outboundDepartureDateTo=${outTo}`
+ `&inboundDepartureDateFrom=${inFrom}&inboundDepartureDateTo=${inTo}`
+ `&market=fr-fr&adultPaxCount=${state.prefs?.adults||1}&currency=EUR&limit=1
try{
const r = await fetch(url);
if(!r.ok) throw new Error('HTTP ' + r.status);
const d = await r.json();
const fares = (d.fares||[]).filter(f=>f.outbound && f.inbound)
.sort((a,b)=>(a.summary?.price?.value??9e9)-(b.summary?.price?.value??9e9))
if(!fares.length){
zone.innerHTML = errHTML(`Aucun vol Ryanair ${from} → ${to} sur cette p rio
return;
}
zone.innerHTML = `<h3 style="margin-bottom:10px">Meilleurs allers-retours tro
const o = f.outbound, b = f.inbound;
const dOut = o.departureDate.slice(0,10), dIn = b.departureDate.slice(0,10)
const book = `https://www.ryanair.com/fr/fr/trip/flights/select?adults=${st
return `<div class="item">
<div class="emo">${i===0?'  ':  }</div>
<div style="flex:1">
<h4>${esc(frDate(o.departureDate))} → ${esc(frDate(b.departureDate))}</
<p>Aller ${esc(frTime(o.departureDate))} (${esc(o.price?.value?.toFixed
${esc(o.departureAirport?.name||from)}    ${esc(o.arrivalAirport?.name||
<a class="tl-loc" href="${esc(book)}" target="_blank" rel="noopener" st
</div>
<div class="side"><span class="tag money" style="font-size:.85rem">   $e
</div>`;
}).join('') + `<p class="hint">Prix r els au moment de la recherche, hors bag
if(fares[0]?.summary?.price?.value){
const v = +fares[0].summary.price.value.toFixed(0);
state.cache.realPrice = `  partir de ${v}   A/R par personne (Ryanair, ${ne
trackPrice(v, 'Ryanair');
save();
}
} catch(e) {
zone.innerHTML = errHTML('API Ryanair injoignable (adblocker ? r seau ?). R e

```

```
}  
}
```

```
/* --- ✈ Ryanair : calendrier des prix du mois (aller simple / jour) --- */  
async function ryCalendar(){  
  const zone = $('#zoneRy');  
  const from = $('#ryFrom').value.trim().toUpperCase().replace(/^[A-Z]/g, '').slice(0,2)  
  const to = $('#ryTo').value.trim().toUpperCase().replace(/^[A-Z]/g, '').slice(0,2)  
  const date = $('#ryDate').value || ryDefaultDate();  
  if(from.length!==3 || to.length!==3){ toast('Renseigne 2 codes IATA'); return;  
  zone.innerHTML = loaderHTML('Chargement du calendrier des prix...');  
  const month = date.slice(0,7) + '-01';  
  try{  
    const r = await fetch(`https://services-api.ryanair.com/farfn/v4/oneWayFares`)  
    if(!r.ok) throw new Error('HTTP ' + r.status);  
    const d = await r.json();  
    const days = (d.outbound?.fares||[]).filter(f=>!f.unavailable && f.price);  
    if(!days.length){ zone.innerHTML = errHTML(`Aucun vol ${from} → ${to} ce mois`);  
    const min = Math.min(...days.map(f=>f.price.value));  
    zone.innerHTML = `

### >17 Aller simple ${esc(from)} <div style="display:flex;flex-wrap:wrap;gap:7px">` + days.map(f=>{ const best = f.price.value === min; return ` ` <div style="font-size:.68rem;color:var(--txt-2)">${esc(frDate(f.day))}</div> <div style="font-family:'Sora';font-weight:900;font-size:.9rem;color:${best?'green':'red'}>` <div style="font-size:.6rem;color:var(--danger)">complet</div> </div>`; }).join('') + `</div><p class="hint">💚 = jour le moins cher du mois (${min.toFixed(2)} €` }catch(e){ zone.innerHTML = errHTML('Calendrier indisponible – API Ryanair injoignable.') } }


```

```
/* --- 🌐 Aviasales (Travelpayouts) : prix réels TOUTES compagnies --- */  
const AIRLINES = {  
  AF:'Air France', U2:'easyJet', FR:'Ryanair', TO:'Transavia France', HV:'Transavia',  
  VY:'Vueling', W6:'Wizz Air', W4:'Wizz Air Malta', LH:'Lufthansa', KL:'KLM', BA:'British Airways',  
  IB:'Iberia', AZ:'ITA Airways', TP:'TAP Portugal', LX:'Swiss', OS:'Austrian', SN:'Scandinavian Airlines',  
  EW:'Eurowings', SK:'SAS', AY:'Finnair', LO:'LOT', A3:'Aegean', PC:'Pegasus', TK:'Turkish Airlines',  
  EK:'Emirates', QR:'Qatar Airways', EY:'Etihad', AT:'Royal Air Maroc', TU:'Tunisair',  
  DY:'Norwegian', D8:'Norwegian', EI:'Aer Lingus', UX:'Air Europa', EN:'Air Dolomiti',  
  XK:'Air Corsica', BF:'French Bee', SS:'Corsair', TX:'Air Caraïbes', ZB:'Air Albania',  
};  
const airlineName = c => AIRLINES[c] || c || '-';
```

```

async function tpSearch(){
  const zone = $('#zoneRy');
  const token = tpKey();
  if(!token){
    zone.innerHTML = errHTML('Token Travelpayouts absent de config.js.');
```

return;

```

  }
  const from = $('#ryFrom').value.trim().toUpperCase().replace(/[^A-Z]/g, '').slice(0, 2);
  const to = $('#ryTo').value.trim().toUpperCase().replace(/[^A-Z]/g, '').slice(0, 2);
  const date = $('#ryDate').value || ryDefaultDate();
  const stay = +$('#ryStay').value;
  if(from.length<2 || to.length<2){ toast('Renseigne 2 codes IATA'); return; }
  zone.innerHTML = loaderHTML('Interrogation Aviasales – toutes compagnies...');
  const ret = addDays(date, stay);
  const base = `https://api.travelpayouts.com/aviasales/v3/prices_for_dates?origin=${from}&destination=${to}&departure_at=${date.toISOString().slice(0, 10)}&return_at=${ret.toISOString().slice(0, 10)}&currency=eur&direct=false&one_way=false&unique=false&sorting=price`;
  try{
    /* 1er essai : dates précises · 2e essai : mois entier (le cache Aviasales est utilisé)
    let r = await fetch(`${base}&departure_at=${date.toISOString().slice(0, 10)}&return_at=${ret.toISOString().slice(0, 10)}`);
    let d = r.ok ? await r.json() : null;
    let broad = false;
    if(!d || !d.success || !(d.data||[]).length){
      broad = true;
      r = await fetch(`${base}&departure_at=${date.toISOString().slice(0, 10)}&return_at=${ret.toISOString().slice(0, 10)}`);
      d = r.ok ? await r.json() : null;
    }
    if(!d || d.success === false){
      const err = d?.error || '';
      zone.innerHTML = errHTML(/token/i.test(err) ? 'Token Travelpayouts invalide' : err);
      return;
    }
    const rows = (d.data||[]).slice(0, 10);
    if(!rows.length){
      zone.innerHTML = errHTML(`Aucun prix en cache pour ${from} → ${to}. L'API Aviasales est utilisée`);
      return;
    }
    zone.innerHTML = `<h3 style="margin-bottom:10px">🌐 Toutes compagnies – ${esc(from)} → ${esc(to)}</h3>
    <table>
      <tr>
        <th>Compagnie</th>
        <th>Aller</th>
        <th>Retour</th>
      </tr>
      <tr>
        <td>
          <div class="item">
            <div class="emo" style="display:flex;align-items:center">
              <div style="flex:1">
                <h4>${esc(airlineName(f.airline))}</h4>
                <p>Aller ${esc(frDate(dep))} à ${esc(frTime(dep))}</p>
              </div>
              <div style="flex:1">
                <p>Retour ${esc(frDate(ret))} à ${esc(frTime(ret))}</p>
              </div>
            </div>
          </td>
        </tr>
      </table>
    `;
  } catch (err) {
    zone.innerHTML = errHTML(err);
  }
}
```



```

        <a class="tl-loc" href="${esc(link)}" target="_blank" rel="noopener"
    </div>
    <div class="side"><span class="tag money" style="font-size:.85rem"><img alt="Euro symbol" data-bbox="855 85 875 105"/> $
    </div>`;
    }).join('') +
    `<p class="hint">Prix issus du cache Aviasales (recherches réelles des dern
if(rows[0]?.price){
    state.cache.realPrice = `à partir de ${rows[0].price} € A/R par personne (t
    trackPrice(+rows[0].price, 'toutes compagnies');
    save();
}
}catch(e){
    zone.innerHTML = errHTML('API Aviasales injoignable (adblocker ? réseau ?).')
}
}

/* ---  Deutsche Bahn : horaires réels --- */
const DB_API = 'https://v6.db.transport.rest';
async function dbStation(q){
    const r = await fetch(`${DB_API}/locations?query=${encodeURIComponent(q)}&resul
    if(!r.ok) throw new Error('HTTP ' + r.status);
    const d = await r.json();
    return d[0] || null;
}
async function dbSearch(){
    const zone = $('#zoneDb');
    const qFrom = $('#dbFrom').value.trim(), qTo = $('#dbTo').value.trim();
    const when = $('#dbWhen').value;
    if(!qFrom || !qTo){ toast('Renseigne les 2 gares'); return; }
    zone.innerHTML = loaderHTML('Recherche des gares...');
    try{
        const [a, b] = await Promise.all([dbStation(qFrom), dbStation(qTo)]);
        if(!a || !b){ zone.innerHTML = errHTML(`Gare introuvable : ${!a?qFrom:qTo}. E
        zone.innerHTML = loaderHTML(`${a.name} → ${b.name}...`);
        const dep = when ? new Date(when).toISOString() : new Date().toISOString();
        const r = await fetch(`${DB_API}/journeys?from=${a.id}&to=${b.id}&departure=$
        if(!r.ok) throw new Error('HTTP ' + r.status);
        const d = await r.json();
        const js = d.journeys || [];
        if(!js.length){ zone.innerHTML = errHTML('Aucun trajet trouvé dans le réseau
        zone.innerHTML = `<h3 style="margin-bottom:10px"><img alt="Train icon" data-bbox="625 815 645 835"/> ${esc(a.name)} → ${esc(b
            const legs = (j.legs||[]).filter(l=>!l.walking);
            if(!legs.length) return '';
            const dep0 = legs[0], arrN = legs[legs.length-1];
            const durMs = new Date(arrN.arrival||arrN.plannedArrival) - new Date(dep0.d
            const dur = `${Math.floor(durMs/36e5)}h${String(Math.round(durMs%36e5/6e4))
            const changes = legs.length - 1;

```

```

const lines = legs.map(l=>l.line?.name).filter(Boolean).join(' → ');
const price = j.price?.amount ? `${j.price.amount.toFixed(2)} ${j.price.cur
const delay = dep0.departureDelay ? Math.round(dep0.departureDelay/60) : 0;
return `
```

```

"quartiers":[
  {"nom":"...", "emoji":"un emoji", "pourquoi":"2 phrases : ambiance + pour qui c'
],
"type_conseille":"1 phrase : hôtel / appart / auberge selon le profil et pourquoi
"conseils":["3 conseils réservation : quand réserver, arnaques à éviter, quartier
}
Donne exactement 3 quartiers.`;
try{
  const d = await gemini(prompt);
  state.cache.stay = d; save();
  renderStay(d);
}catch(e){
  if(e.message!=='NO_KEY') zone.innerHTML = errHTML('Recherche logement impossi
}
}

function renderStay(d){
  const t = state.trip;
  const booking = q => `https://www.booking.com/searchresults.fr.html?ss=${encodeURIComponent(q)}`;
  const airbnb = q => `https://www.airbnb.fr/s/${encodeURIComponent(q)}/homes`;
  const hostel = q => `https://www.hostelworld.com/s?q=${encodeURIComponent(q)}`;
  $('#zoneStay').innerHTML = `
    <div class="item"><div class="emo">🔍</div><p style="margin-top:4px"><strong>
    <h3 style="margin:16px 0 10px">Les 3 quartiers où viser</h3>
    ${ (d.quartiers||[]).map(q=>`
      <div class="item">
        <div class="emo">${esc(q.emoji||'🏠')}</div>
        <div style="flex:1">
          <h4>${esc(q.nom)} <span class="tag" style="margin-left:6px">${esc(q.id)}
          <p>${esc(q.pourquoi)}</p>
          <div class="row" style="margin-top:9px">
            <a class="btn sm" href="${booking(q.nom + ', ' + t.nom)}" target="_bl
            <a class="btn sm ghost" href="${airbnb(q.nom + ', ' + t.nom)}" target
          </div>
        </div>
        <div class="side"><span class="tag money">💰 ${esc(q.prix_nuit)}</span></div>
      </div>`).join('')
    <h3 style="margin:16px 0 8px">Conseils réservation</h3>
    ${ (d.conseils||[]).map(c=>`<div class="item"><div class="emo">💡</div><p styl
    <div class="divider"></div>
    <div class="row">
      <a class="btn" href="${booking(t.nom)}" target="_blank" rel="noopener">🏠 E
      <a class="btn ghost" href="${airbnb(t.nom)}" target="_blank" rel="noopener"
      <a class="btn ghost" href="${hostel(t.nom)}" target="_blank" rel="noopener"
    </div>`;
  }

```

```

/* =====
ÉTAPE 4 – SUR PLACE
===== */

function setMap(q) {
  $('#mapFrame').src = `https://maps.google.com/maps?q=${encodeURIComponent(q)}&h
}
window.setMap = setMap;

/* Délégation : tout élément avec data-loc met à jour la carte */
document.addEventListener('click', e => {
  const el = e.target.closest('[data-loc]');
  if(!el) return;
  const q = el.dataset.loc + (state.trip ? ', ' + state.trip.nom : '');
  setMap(q);
  toast('📍 ' + el.dataset.loc);
  $('#mapFrame').scrollIntoView({behavior:'smooth', block:'center'});
});

/* --- sous-onglets --- */
const TAB_PANELS = {iti:'#pIti', food:'#pFood', shop:'#pShop', spec:'#pSpec', bag
$('#subtab').forEach(el => el.onclick = () => {
  $('#subtab').forEach(x=>x.classList.remove('on')); el.classList.add('on');
  Object.entries(TAB_PANELS).forEach(([k,sel]) => $(sel).classList.toggle('hidden
});

/* =====
ITINÉRAIRE (Gemini · heavy)
===== */

function itiPrompt(day) {
  const t = state.trip;
  const pace = $('#itiPace').value, wish = $('#itiWish').value.trim();
  const start = $('#itiStart').value, end = $('#itiEnd').value, move = $('#itiMov
  return `Tu es Acolite, guide local expert de ${t.nom} (${t.pays}). ${ctx()}
  Construis le programme du JOUR ${day} du séjour.
  - Rythme : ${pace} · Journée de ${start} à ${end} · Déplacements : ${move}
  ${wish ? '- Envie particulière : '+wish : ''}
  Programme RÉALISTE : horaires cohérents, temps de trajet inclus, pauses repas.
  Si jour > 1, varie par rapport aux grands classiques du jour 1.
  Réponds UNIQUEMENT en JSON :
  {
    "titre_journee":"thème du jour",
    "etapes":[
      {"heure":"09:00","titre":"...", "description":"1-2 phrases concrètes avec un vr
    ]
  }
  Entre 6 et 9 étapes.`;

```

```

}

function timelineHTML(d) {
  const icons = {visite:'🏠', repas:'🍽️', pause:'☕', trajet:'🚶'};
  return `<div class="timeline">
    ${ (d.etapes||[]).map((e,i)=>`
      <div class="tl-item" style="animation-delay:${i*0.06}s">
        <div class="tl-time">${esc(e.heure)} ${icons[e.type]} '📍'</div>
        <div class="tl-title">${esc(e.titre)}</div>
        <div class="tl-desc">${esc(e.description)}</div>
        ${e.lieu ? `<span class="tl-loc" data-loc="${esc(e.lieu)}">📍 Voir sur la
        </div>`)}.join('')}
    </div>`;
}

const _e7 = $('#btnIti'); if(_e7) _e7.onclick = async () => {
  const zone = $('#zoneIti');
  const day = $('#itiDay').value;
  zone.innerHTML = loaderHTML('Construction de ton programme...');
  $('#btnIti').disabled = true;
  try{
    const d = await gemini(itiPrompt(day));
    zone.innerHTML = `<h3 style="margin-bottom:14px">🌟 Jour ${esc(day)} - ${esc(
  )}catch(e){
    if(e.message!=='NO_KEY') zone.innerHTML = errHTML('Génération impossible, rée
  }
  $('#btnIti').disabled = false;
};

/* --- tout le séjour d'un coup --- */
function daysFromPrefs(){
  const d = (state.prefs?.days || '').toLowerCase();
  if(d.includes('week-end')) return 3;
  if(d.includes('deux')) return 14;
  if(d.includes('trois')) return 14;
  return 7;
}

const _e8 = $('#btnItiAll'); if(_e8) _e8.onclick = async () => {
  const zone = $('#zoneItiAll');
  const t = state.trip;
  const n = Math.min(daysFromPrefs(), 10);
  zone.innerHTML = loaderHTML(`Planification des ${n} jours... (ça peut prendre ~30
  $('#btnItiAll').disabled = true;
  const pace = $('#itiPace').value, move = $('#itiMove').value;
  const prompt = `Tu es Acolite, guide local expert de ${t.nom} (${t.pays}). ${ct
  Construis le programme COMPLET du séjour sur ${n} jours. Rythme : ${pace}. Déplac

```

Chaque jour a un thème différent, sans répéter les lieux. Jour 1 = incontournable
Réponds UNIQUEMENT en JSON :

```
{
  "jours":[
    {"jour":1,"titre":"thème","etapes":[{"heure":"09:00","titre":"...", "descriptio
  ]
}
4 à 6 étapes par jour, pour rester lisible.`;
try{
  const d = await gemini(prompt, true, 8192);
  state.cache.fullPlan = d; save();
  renderFullPlan(d);
}catch(e){
  if(e.message!=='NO_KEY') zone.innerHTML = errHTML('Trop gros pour cette fois
}
$('#btnItiAll').disabled = false;
};

function renderFullPlan(d){
  $('#zoneItiAll').innerHTML = `<div class="divider"></div><h3 style="margin-bott
(d.jours||[]).map((j,i)=>`
  <div class="acc ${i===0?'open':''}">
    <div class="acc-head" data-acc>
      Jour ${esc(j.jour)} - ${esc(j.titre)} <span class="arr"></span>
    </div>
    <div class="acc-body">${timelineHTML(j)}</div>
  </div>`).join('');
}

/* =====
  RESTOS (Gemini · heavy - connaissance locale précise)
===== */
const _e9 = $('#btnFoodGo'); if(_e9) _e9.onclick = () => { delete state.cache.foo

async function loadFood(){
  const zone = $('#zoneFood');
  if(state.cache.food){ renderFood(state.cache.food); return; }
  zone.innerHTML = loaderHTML('Dégustation en cours...');
  const t = state.trip;
  const fb = $('#foodBud').value, ft = $('#foodType').value;
  const prompt = `Tu es Acolite, fin gourmet local de ${t.nom} (${t.pays}). ${ctx
  ${fb ? 'Budget resto souhaité : ' + fb + '.' : ''}
  ${ft ? 'Envie : ' + ft + '.' : ''}
  Réponds UNIQUEMENT en JSON :
  {"restos":[
    {"nom":"nom réel et connu du resto","emoji":"1 emoji plat","style":"ex: trattori
  ]}
```

```

Exactement 5 restos VARIÉS, vraiment réputés à ${t.nom}.`;
    try{
        const d = await gemini(prompt);
        state.cache.food = d; save();
        renderFood(d);
    }catch(e){ if(e.message!=='NO_KEY') zone.innerHTML = errHTML('Chargement des re
}
function renderFood(d){
    const t = state.trip;
    $('#zoneFood').innerHTML = (d.restos||[]).map(r=>`
        <div class="item">
            <div class="emo">${esc(r.emoji||'🍲')}</div>
            <div style="flex:1">
                <h4>${esc(r.nom)} <span class="tag" style="margin-left:6px">${esc(r.style
                <p><strong>À commander :</strong> ${esc(r.plat_star)} – ${esc(r.pourquoi)
                <div class="row" style="margin-top:8px">
                    <span class="tl-loc" data-loc="${esc(r.nom)}">📍 Carte</span>
                    <a class="tl-loc" href="https://www.google.com/maps/search/${encodeURIComponent
                </div>
            </div>
            <div class="side"><span class="tag money">${esc(r.budget)}</span><p style="
        </div>`).join('');
    }

/* =====
    COURSES (light → Groq)
===== */
async function loadShop(){
    const zone = $('#zoneShop');
    if(state.cache.shop){ renderShop(state.cache.shop, state.cache.shopVia); return
    zone.innerHTML = loaderHTML('Repérage des supermarchés...');
    const t = state.trip;
    const prompt = `Tu es Acolite, expert du quotidien à ${t.nom} (${t.pays}). ${ct
Réponds UNIQUEMENT en JSON :
{
    "supermarches": [{"nom": "chaîne réelle du pays", "niveau": "discount|standard|premi
    "marches": [{"nom": "marché local réel", "quand": "jours/horaires", "pourquoi": "1 phr
    "budget_conseils": ["3 conseils concrets pour manger pas cher sur place"]
}
3 supermarchés, 1-2 marchés.`;
    try{
        const {data, via} = await ai('light', prompt);
        state.cache.shop = data; state.cache.shopVia = via; save();
        renderShop(data, via);
    }catch(e){ if(e.message!=='NO_KEY') zone.innerHTML = errHTML('Chargement imposs
}
function renderShop(d, via){

```

```

const lvl = {discount:'🏷️ Discount', standard:'🛒 Standard', premium:'🌟 Premi
$('#zoneShop').innerHTML = `
<h3 style="margin-bottom:10px">Supermarchés ${badge(via)}</h3>
${(d.supermarches||[]).map(s=>`
  <div class="item"><div class="emo">🛒</div>
  <div style="flex:1"><h4>${esc(s.nom)} <span class="tag cyan" style="margi
  <div class="side"><span class="tl-loc" data-loc="${esc(s.nom)}">📍</span>
</div>`)}.join('')}
<h3 style="margin:14px 0 10px">Marchés locaux</h3>
${(d.marches||[]).map(m=>`
  <div class="item"><div class="emo">🍲</div>
  <div style="flex:1"><h4>${esc(m.nom)}</h4><p>${esc(m.pourquoi)} · <strong
  <div class="side"><span class="tl-loc" data-loc="${esc(m.nom)}">📍</span>
</div>`)}.join('')}
<h3 style="margin:14px 0 10px">Manger malin</h3>
${(d.budget_conseils||[]).map(c=>`<div class="item"><div class="emo">💡</div>
}

/* =====
SPÉCIALITÉS (light → Groq)
===== */
async function loadSpec(){
  const zone = $('#zoneSpec');
  if(state.cache.spec){ renderSpec(state.cache.spec, state.cache.specVia); return
  zone.innerHTML = loaderHTML('Enquête gourmande...');
  const t = state.trip;
  const prompt = `Tu es Acolite, passionné de gastronomie de ${t.nom} (${t.pays})
Réponds UNIQUEMENT en JSON :
{
  "specialites":[{"nom":"plat/produit local","emoji":"1 emoji","description":"c'es
  "conseils_locaux":["3-4 conseils culture food locale : usages à table, pourboire
}
5-6 spécialités emblématiques.`;
  try{
    const {data, via} = await ai('light', prompt);
    state.cache.spec = data; state.cache.specVia = via; save();
    renderSpec(data, via);
  }catch(e){ if(e.message!=='NO_KEY') zone.innerHTML = errHTML('Chargement imposs
}
function renderSpec(d, via){
  $('#zoneSpec').innerHTML = `
  <div style="margin-bottom:10px">${badge(via)}</div>
  ${((d.specialites||[]).map(s=>`
    <div class="item"><div class="emo">${esc(s.emoji||'🍲')}</div>
    <div style="flex:1"><h4>${esc(s.nom)}</h4><p>${esc(s.description)}<br><st
    <div class="side"><span class="tag money">${esc(s.prix)}</span></div>
  </div>`)}.join('')}

```



```

<h3 style="margin:14px 0 10px">Les codes locaux 🍷</h3>
  ${ (d.conseils_locaux||[]).map(c=><div class="item"><div class="emo">💡</div>
}

/* =====
  VALISE (light → Groq) – checklist persistée
===== */
async function loadBag(){
  const zone = $('#zoneBag');
  if(state.cache.bag){ renderBag(state.cache.bag, state.cache.bagVia); return; }
  zone.innerHTML = loaderHTML('Préparation de ta checklist...');
  const t = state.trip;
  const R = state.cache._real || {};
  const d = stayDates();
  const nuits = d ? Math.max(1, Math.round((new Date(d.out) - new Date(d.in)) / 8
  const prompt = `Tu es Acolite. ${ctx()}
Génère la checklist valise idéale pour ce voyage à ${t.nom} (${t.pays}).
${R.meteo ? `MÉTÉO RÉELLE MESURÉE (adapte les vêtements À CES CHIFFRES, ne les co
${nuits ? `DURÉE EXACTE : ${nuits} nuit(s) – dimensionne les quantités (nombre de
${state.cache.plan?.programme?.length ? `PROGRAMME PRÉVU (prévois les tenues adap
${state.prefs?.kids ? `Voyage AVEC ${state.prefs.kids} enfant(s) : ajoute le néce
Adapte aux activités probables et au profil.
Réponds UNIQUEMENT en JSON :
{"categories":[
  {"nom":"ex: Vêtements","emoji":"1 emoji","items":["6-10 items courts et concrets
]}
4-5 catégories (Vêtements, Documents & argent, Tech, Santé & trousse, Spécifique
  try{
    const {data, via} = await ai('light', prompt);
    state.cache.bag = data; state.cache.bagVia = via; save();
    renderBag(data, via);
  }catch(e){ if(e.message!=='NO_KEY') zone.innerHTML = errHTML('Chargement imposs
}
function renderBag(d, via){
  $('#bagBadge').style.display = via==='groq' ? '' : 'none';
  let html = '';
  (d.categories||[]).forEach((c,ci)=>{
    html += <h3 style="margin:${ci?14:0}px 0 9px">${esc(c.emoji||'📦')} ${esc(c.
    (c.items||[]).forEach((it,ii)=>{
      const k = ci + '_' + ii;
      html += <div class="check ${state.checklist[k]?'done':''}" data-ck="${k}">
        <div class="box">${state.checklist[k]?'✓':''}</div><span>${esc(it)}</span>
        </div>`;
    });
  });
  $('#zoneBag').innerHTML = html;
  updateBagProg();

```

```

}
document.addEventListener('click', e => {
  const el = e.target.closest('[data-ck]');
  if(!el) return;
  const k = el.dataset.ck;
  state.checklist[k] = !state.checklist[k];
  save();
  el.classList.toggle('done', state.checklist[k]);
  el.querySelector('.box').textContent = state.checklist[k] ? '✓' : '';
  updateBagProg();
  const boxes = $$('#zoneBag .check');
  if(boxes.length && boxes.every(b => b.classList.contains('done'))){ confetti();
});
function updateBagProg(){
  const total = $$('#zoneBag .check').length;
  const done = $$('#zoneBag .check.done').length;
  $('#bagProg').style.width = total ? Math.round(done/total*100)+'%' : '0%';
  const cnt = $('#bagCnt');
  if(cnt) cnt.textContent = total ? `${done}/${total}` : '';
}

/* =====
  PHRASES (light → Groq)
===== */
async function loadTalk(){
  const zone = $('#zoneTalk');
  if(state.cache.talk){ renderTalk(state.cache.talk, state.cache.talkVia); return
  zone.innerHTML = loaderHTML('Traduction en cours...');
  const t = state.trip;
  const prompt = `Tu es Acolite. Destination : ${t.nom} (${t.pays}), langue local
Si la langue locale est le français, donne plutôt les expressions/argot local typ
Réponds UNIQUEMENT en JSON :
{"langue":"nom de la langue","phrases":[
  {"fr":"phrase en français","local":"traduction en langue locale","pron":"prononc
]}
12 phrases : bonjour/merci/svp, se présenter, commander au resto, demander l'addi
  try{
    const {data, via} = await ai('light', prompt);
    state.cache.talk = data; state.cache.talkVia = via; save();
    renderTalk(data, via);
  }catch(e){ if(e.message!=='NO_KEY') zone.innerHTML = errHTML('Chargement imposs
}
function renderTalk(d, via){
  $('#talkBadge').style.display = via==='groq' ? '' : 'none';
  $('#zoneTalk').innerHTML = `

### 


```

```

        <div class="fr">${esc(p.fr)}</div>
        <div class="loc">${esc(p.local)}</div>
        <div class="pron">👤 ${esc(p.pron)}</div>
    </div>`).join('');
}

/* =====
    BUDGET (Gemini · heavy) + tracker dépenses
===== */
const BUD_COLORS = ['#00F0FF','#A855F7','#FFE600','#FF6B00','#22C55E','#EF4444'];

async function loadBudget(){
    const zone = $('#zoneBud');
    if(state.cache.bud){ renderBudget(state.cache.bud); return; }
    zone.innerHTML = loaderHTML('Calcul du budget...');
    const t = state.trip;
    const prompt = `Tu es Acolite, expert budget voyage. ${ctx()}
Estime le budget TOTAL réaliste par personne pour ce séjour à ${t.nom} (${t.pays}
Réponds UNIQUEMENT en JSON :
{
    "total_estime":nombre entier en euros,
    "postes":[
        {"nom":"Transport aller-retour","montant":nombre,"detail":"1 phrase"},
        {"nom":"Logement","montant":nombre,"detail":"1 phrase"},
        {"nom":"Repas","montant":nombre,"detail":"1 phrase"},
        {"nom":"Activités & visites","montant":nombre,"detail":"1 phrase"},
        {"nom":"Transports sur place","montant":nombre,"detail":"1 phrase"},
        {"nom":"Extra & imprévus","montant":nombre,"detail":"1 phrase"}
    ],
    "astuces":["3 astuces concrètes pour réduire ce budget de 20-30%"]
}`;
    try{
        const d = await gemini(prompt);
        state.cache.bud = d; save();
        renderBudget(d);
    }catch(e){ if(e.message!=='NO_KEY') zone.innerHTML = errHTML('Calcul impossible
}
function renderBudget(d){
    const total = d.total_estime || (d.postes||[]).reduce((a,p)=>a+(+p.montant||0),
    const sum = (d.postes||[]).reduce((a,p)=>a+(+p.montant||0),0) || 1;
    $('#zoneBud').innerHTML = `
        <div class="row" style="justify-content:space-between">
            <div><div class="hint" style="margin:0">ESTIMATION TOTALE / PERSONNE <span
            <div class="spend-total">${total} €</div></div>
        </div>
        <div class="bud-bar">${(d.postes||[]).map((p,i)=>`<i style="width:${(+p.monta
        <div class="legend">${(d.postes||[]).map((p,i)=>`<span><span class="sw" style

```

```

    ${d.postes||[]}.map((p,i)=><div class="item"><div class="emo" style="font-s
<h3 style="margin:14px 0 8px">Réduire la note 📉</h3>
    ${d.astuces||[]}.map(a=><div class="item"><div class="emo">💡</div><p style
}

/* --- dépenses réelles --- */
const _e10 = $('#btnSpend'); if(_e10) _e10.onclick = () => {
  const label = $('#spLabel').value.trim() || 'Dépense';
  const amount = parseFloat($('#spAmount').value);
  if(isNaN(amount) || amount <= 0){ toast('Entre un montant valide 📅'); return;
  state.spends.push({ label, amount, ts: Date.now() });
  save();
  $('#spLabel').value = ''; $('#spAmount').value = '';
  renderSpends();
};
function renderSpends(){
  const zone = $('#zoneSpends');
  if(!zone) return;
  const total = state.spends.reduce((a, s) => a + s.amount, 0);
  /* budget de référence : celui du plan IA en priorité, sinon l'estimation déta
  const A = (state.prefs?.adults || 1) + (state.prefs?.kids || 0);
  const est = (state.cache.plan?.budget?.total || 0) * A || state.cache.bud?.tota
  const d = stayDates();
  let bar = '';
  if(est){
    const pct = Math.round(total / est * 100);
    /* rythme attendu : où devrais-tu en être aujourd'hui ? */
    let attendu = null;
    if(d){
      const dep = new Date(d.in), fin = new Date(d.out), now = new Date();
      const nDays = Math.max(1, Math.round((fin - dep) / 86400000));
      const passed = Math.min(nDays, Math.max(0, Math.ceil((now - dep) / 86400000
      if(passed > 0 && now <= fin) attendu = Math.round(est * passed / nDays);
    }
    const derive = attendu ? total - attendu : 0;
    const alerte = attendu && Math.abs(derive) > est * 0.08;
    bar = `
      <div class="row" style="justify-content:space-between;align-items:center;ma
        <div class="spend-total">${total.toFixed(2)} €</div>
        <span class="tag ${pct > 100 ? 'money' : 'cyan'}">${pct}% du budget (${es
      </div>
      <div class="progress"><i style="width:${Math.min(100, pct)}%;background:${p
      ${alerte ? `<p class="hint" style="margin-top:8px;font-weight:800;color:${d
        ${derive > 0
          ? `⚠️ Tu dépenses plus vite que prévu : ${Math.round(derive)} € au-dess
          : `✅ Tu es en dessous du rythme prévu : ${Math.abs(Math.round(derive))
      ${pct > 100 ? `<p class="hint" style="margin-top:6px;font-weight:800;color:

```

```

    }
    if(!state.spends.length){
      zone.innerHTML = bar + `
```

```

}
function renderInfo(d, via){
  $('#infoBadge').style.display = via==='groq' ? '' : 'none';
  const rows = [
    [, 'Formalités', d.formalites], [, 'Santé', d.sante], [, 'Sécurité',
    ['$¥', 'Argent', d.argent], [, 'Électricité', d.electricite], [, 'Téléphone',
    [, 'Urgences', d.urgences], [, 'Meilleur moment', d.meilleur_moment]
  ];
  $('#zoneInfo').innerHTML = rows.filter(r=>r[2]).map(r=>`
    <div class="item"><div class="emo">${r[0]}</div><div><h4>${r[1]}</h4><p>${esc
}

/* =====
EXPORT .md
===== */
const _e11 = $('#btnExport'); if(_e11) _e11.onclick = () => {
  if(!state.trip){ toast('Choisis d'abord une destination'); return; }
  const t = state.trip, p = state.prefs || {}, c = state.cache;
  let md = `#  Voyage Acolite - ${t.nom}, ${t.pays}\n\n`;
  md += `- **Départ :** ${p.from||''}\n- **Durée :** ${p.days||''}\n- **Période :
  if(c.plan){
    const pl = c.plan;
    md += `\n##  Plan Acolite${state.planOk ? ' (validé )' : ''}\n`;
    md += `- **Transport choisi :** ${pl.transport?.mode||''} - ${pl.transport?.p
    md += `- **Logement :** ${pl.logement?.type||''} à ${pl.logement?.quartier||'
    md += `- **Budget total :** ${pl.budget?.total||''} €/pers - ${pl.budget?.rep
    (pl.programme||[]).forEach(j=> md += `- Jour ${j.jour} : ${j.resume}\n`);
    md += `-  ${pl.conseil_cle||''}\n`;
  }
}
const tr = c['transport_'+state.mode];
if(tr){
  md += `\n##  Transport (${state.mode})\n`;
  Object.entries(tr).forEach(([k,v])=>{
    if(Array.isArray(v)) md += `- **${k} :**\n` + v.map(x=>` - ${x}`).join('\n
    else md += `- **${k} :** ${v}\n`;
  });
}
if(c.stay){
  md += `\n##  Logement\n${c.stay.type_conseille||''}\n`;
  (c.stay.quartiers||[]).forEach(q=> md += `- **${q.nom}** (${q.prix_nuit}) - $
}
if(c.fullPlan){
  md += `\n##  Programme\n`;
  (c.fullPlan.jours||[]).forEach(j=>{
    md += `\n### Jour ${j.jour} - ${j.titre}\n`;
    (j.etapes||[]).forEach(e=> md += `- **${e.heure}** ${e.titre}${e.lieu?'_( $

```

```

    });
}
if(c.food){ md += `\\n## 🍽 Restos\\n`; (c.food.restos||[]).forEach(r=> md += `
if(c.spec){ md += `\\n## 🍷 Spécialités\\n`; (c.spec.specialites||[]).forEach(s=>
if(c.shop){ md += `\\n## 🛒 Courses\\n`; (c.shop.supermarches||[]).forEach(s=> md
if(c.bag){ md += `\\n## 🧳 Valise\\n`; (c.bag.categories||[]).forEach(cat=>{ md +
if(c.talk){ md += `\\n## 🗣 Phrases utiles (${c.talk.langue||''})\\n`; (c.talk.ph
if(c.bud){ md += `\\n## 💰 Budget estimé : ${c.bud.total_estime} €/pers\\n`; (c.b
if(c.act){ md += `\\n## 🎡 Activités\\n`; (c.act.activites||[]).forEach(a=> md +=
if(c.info){
    md += `\\n## 📍 Infos pratiques\\n`;
    Object.entries(c.info).forEach(([k,v])=> md += ` - **${k} : ** ${v}\\n`);
}
if(state.resas.length){
    md += `\\n## 📅 Réservations\\n`;
    state.resas.forEach(r=> md += ` - ${r.type} - ${r.ref}${r.link?` (${r.link})`:
}
if(state.notes.trim()){ md += `\\n## 📝 Notes\\n${state.notes}\\n`; }
if(state.spends.length){
    const tot = state.spends.reduce((a,s)=>a+s.amount,0);
    md += `\\n## 🇪🇺 Dépenses réelles : ${tot.toFixed(2)} €\\n`;
    state.spends.forEach(s=> md += ` - ${s.label} : ${s.amount.toFixed(2)} €\\n`);
}
md += `\\n---\\n Généré par Acolite ✦ Gemini ⚡ Groq\\n`;
const blob = new Blob([md], {type:'text/markdown'});
const a = document.createElement('a');
a.href = URL.createObjectURL(blob);
a.download = `acolite-${t.nom.toLowerCase().replace(/^[^a-z0-9]+/g, '-')}.md`;
a.click();
URL.revokeObjectURL(a.href);
toast('Voyage exporté 📄');
};

/* =====
    ACTIVITÉS & EXPÉRIENCES (Gemini · heavy)
===== */
async function loadAct(){
    const zone = $('#zoneAct');
    const t = state.trip;
    // liens de résa (toujours dispo)
    const gyg = `https://www.getyourguide.fr/-10/?q=${encodeURIComponent(t.nom)}`;
    const civ = `https://www.civitatis.com/fr/?ns_campaign=acolite#s=${encodeURIComponent
    const tam = `https://www.tripadvisor.fr/Search?q=${encodeURIComponent(t.nom + '
    $('#actLinks').innerHTML = `
        <a class="btn" href="${gyg}" target="_blank" rel="noopener">GetYourGuide</a>
        <a class="btn ghost" href="${civ}" target="_blank" rel="noopener">Civitatis</
        <a class="btn ghost" href="${tam}" target="_blank" rel="noopener">Tripadvisor

```

```

    if(state.cache.act){ renderAct(state.cache.act); return; }
    zone.innerHTML = loaderHTML('Repérage des meilleures activités...');
    const prompt = `Tu es Acolite, guide local de ${t.nom} (${t.pays}). ${ctx()}
Liste les meilleures activités et expériences à vivre sur place, adaptées au prof
Réponds UNIQUEMENT en JSON :
{"activites":[
  {"nom":"nom de l'activité/lieu","emoji":"1 emoji","categorie":"incontournable|na
  ]}
Exactement 7 activités VARIÉES et vraiment marquantes à ${t.nom}.`;
    try{
      const d = await gemini(prompt);
      state.cache.act = d; save();
      renderAct(d);
    }catch(e){ if(e.message!=='NO_KEY') zone.innerHTML = errHTML('Chargement imposs
}
function renderAct(d){
  const t = state.trip;
  const catCol = {incontournable:'money', nature:'', culture:'cyan', sensation:''
  $('#zoneAct').innerHTML = (d.activites||[]).map(a=>`
    <div class="item">
      <div class="emo">${esc(a.emoji||'🤖')}</div>
      <div style="flex:1">
        <h4>${esc(a.nom)} <span class="tag ${catCol[a.categorie]}|''" style="mar
        <p>${esc(a.description)}<br><strong>💡 </strong>${esc(a.astuce)}</p>
        <div class="row" style="margin-top:8px">
          <span class="tl-loc" data-loc="${esc(a.nom)}">📍 Carte</span>
          ${a.reserver==='oui' ? `<a class="tl-loc" href="https://www.getyourguid
        </div>
      </div>
      <div class="side"><span class="tag money">${esc(a.prix)}</span><p style="fo
    </div>`).join('');
}

/* =====
  OUTILS – APIs publiques gratuites, données réelles
  Open-Meteo (géo + météo) · Frankfurter (devises) · Groq (traduction)
===== */
async function loadTools(){
  loadMeteo(); loadTimeAndCurrency(); startCountdown();
}

// --- Géocodage (Open-Meteo geocoding, sans clé) ---
async function geocode(){
  if(state._geo) return state._geo;
  const t = state.trip;
  try{
    const r = await fetch(`https://geocoding-api.open-meteo.com/v1/search?name=${

```



```

    const d = await r.json();
    const g = d.results?.[0];
    if(g){ state._geo = g; return g; }
  }catch(e){}
  return null;
}

// --- Météo 7 jours (Open-Meteo, données réelles gratuites) ---
async function loadMeteo(){
  const zone = $('#zoneMeteo');
  const g = await geocode();
  if(!g){ zone.innerHTML = errHTML('Localisation introuvable pour la météo.');
```

re

```

  try{
    const r = await fetch(`https://api.open-meteo.com/v1/forecast?latitude=${g.la
    const d = await r.json();
    const wmo = c => {
      if(c===0) return ['☀️','Ensoleillé'];
      if(c<=2) return ['☁️','Peu nuageux'];
      if(c===3) return ['☁️','Couvert'];
      if(c<=48) return ['🌫️','Brouillard'];
      if(c<=67) return ['🌧️','Pluie'];
      if(c<=77) return ['❄️','Neige'];
      if(c<=82) return ['🌩️','Averses'];
      if(c<=99) return ['🌩️','Orage'];
      return ['🌡️','-'];
    };
    const days = d.daily.time.map((t,i)=>{
      const w = wmo(d.daily.weather_code[i]);
      const dt = new Date(t);
      const jour = dt.toLocaleDateString('fr-FR',{weekday:'short',day:'numeric'})
      return `<div class="item" style="flex-direction:column;align-items:center;t
        <div style="font-size:.74rem;color:var(--txt-2);text-transform:capitalize
        <div style="font-size:1.6rem;margin:4px 0">${w[0]}</div>
        <div style="font-size:.72rem;color:var(--txt-2)">${w[1]}</div>
        <div style="font-family:'Sora';font-weight:900;margin-top:4px">${Math.rou
        <div style="font-size:.7rem;color:#00F0FF;margin-top:3px;font-weight:800"
        </div>`;
      }).join('');
      zone.innerHTML = `<div style="display:flex;gap:8px;overflow-x:auto;padding-bo
    }catch(e){ zone.innerHTML = errHTML('Météo indisponible pour le moment.');
```

}

```

// --- Heure locale + devises ---
async function loadTimeAndCurrency(){
  const g = await geocode();
  const t = state.trip;
  // heure locale via le timezone renvoyé par la météo

```

```

try{
  const r = await fetch(`https://api.open-meteo.com/v1/forecast?latitude=${g.la
  const d = await r.json();
  const tz = d.timezone;
  const now = new Date();
  const localStr = now.toLocaleTimeString('fr-FR',{hour:'2-digit',minute:'2-dig
  const hereStr = now.toLocaleTimeString('fr-FR',{hour:'2-digit',minute:'2-dig
  const offH = Math.round((d.utc_offset_seconds - now.getTimezoneOffset()*-60)/
  const decal = offH===0 ? 'Même fuseau qu\'en France' : `${offH>0?'+'+'}`+`${off
  $('#zoneTime').innerHTML = `

<div class="emo">
}catch(e){ $('#zoneTime').innerHTML = `


```

```

const _el2 = $('#btnTr'); if(_el2) _el2.onclick = async () => {
  const q = $('#trInp').value.trim();
  if(!q) return;
  const t = state.trip;
  $('#zoneTr').innerHTML = loaderHTML('Traduction...');
  const prompt = `Traduis cette phrase française vers ${t.langue || 'la langue lo
Phrase : "${q}"
Réponds UNIQUEMENT en JSON : {"local":"la traduction","pron":"prononciation phoné
  try{
    const {data, via} = await ai('light', prompt);
    $('#trBadge').style.display = via==='groq' ? '' : 'none';
    $('#zoneTr').innerHTML = `
```

```

const ref = $('#resRef').value.trim();
if(!ref){ toast('Ajoute au moins une référence'); return; }
state.resas.push({ type: $('#resType').value, ref, link: $('#resLink').value.tr
save();
$('#resRef').value = ''; $('#resLink').value = '';
renderResas();
toast('Réservation ajoutée ');
};

function renderResas(){
const zone = $('#zoneRes');
if(!state.resas.length){ zone.innerHTML = `<p class="hint">Aucune réservation e
zone.innerHTML = state.resas.map((r,i)=>`
  <div class="item" style="padding:12px 14px">
    <div class="emo">${esc(r.type.split(' ')[0])}</div>
    <div style="flex:1"><h4>${esc(r.type.replace(/^\S+\s/, ''))}</h4><p>${esc(r.
    <div class="side"><span class="spend-del" data-res="${i}"></span></div>
  </div>`).join('');
}

document.addEventListener('click', e => {
const el = e.target.closest('[data-res]');
if(!el) return;
state.resas.splice(+el.dataset.res, 1);
save(); renderResas();
});

/* =====
  UI GÉNÉRALE
===== */
const _e14 = $('#btnGo'); if(_e14) _e14.onclick = () => { state.propAnswers = [];
const _e15 = $('#btnLucky'); if(_e15) _e15.onclick = () => { state.propAnswers =

const _e16 = $('#btnReset'); if(_e16) _e16.onclick = () => {
  if(!confirm('Repartir de zéro ? (tes clés API sont conservées)')) return;
  localStorage.removeItem(LS_TRIP);
  location.reload();
};

/* =====
  COMPTE - création, connexion, vérification par email
  Stockage local (test) · envoi du code via EmailJS si configuré,
  sinon mode démo (code affiché à l'écran).
===== */
const LS_USER = 'acolite_user';
const LS_AUTH = 'acolite_logged';
const getUser = () => { try{ return JSON.parse(localStorage.getItem(LS_USER)); }c

```

```

const setUser = u => localStorage.setItem(LS_USER, JSON.stringify(u));

async function sha(txt){
  try{
    const b = await crypto.subtle.digest('SHA-256', new TextEncoder().encode(txt))
    return [...new Uint8Array(b)].map(x=>x.toString(16).padStart(2,'0')).join('')
  }catch(e){ /* contexte non sécurisé (file://) → hash simple de secours */
    let h = 0; for(const c of txt){ h = (h*31 + c.charCodeAt(0)) >>> 0; } return
  }
}

function authErr(msg){ const el = $('#authErr'); if(!msg){ el.classList.add('hidd
function authShow(which){
  ['authSignup','authLogin','authVerify'].forEach(id => $('#'+id).classList.toggl
  authErr('');
  $('#authSub').textContent = which==='authSignup' ? "Crée ton compte pour commen
    : which==='authLogin' ? 'Content de te revoir !' : 'Dernière étape : vérifie
}

async function issueCode(u){
  const code = String(Math.floor(100000 + Math.random()*900000));
  u.codeH = await sha('code::' + code);
  u.codeAt = Date.now();
  u.tries = 0;
  delete u.code; /* plus jamais de code en clair dans le stockage */
  setUser(u);
  return code;
}

async function sendVerifyEmail(email, code){
  const ej = CFG.emailjs || {};
  if(ej.publicKey && ej.serviceId && ej.templateId){
    try{
      const r = await fetch('https://api.emailjs.com/api/v1.0/email/send', {
        method:'POST', headers: {'Content-Type':'application/json'},
        body: JSON.stringify({
          service_id: ej.serviceId, template_id: ej.templateId, user_id: ej.publi
          template_params: { to_email: email, code: code }
        })
      });
      if(r.ok){ $('#vfDemo').textContent = ''; toast('📧 Code envoyé à ' + email)
    }catch(e){}
    toast('⚠️ Envoi email échoué — mode démo activé');
  }
  /* Mode démo : pas de service email configuré → code affiché */
  $('#vfDemo').textContent = '📧 Mode démo (EmailJS non configuré dans config.js)
  return false;

```

```

}

const _e17 = $('#goLogin'); if(_e17) _e17.onclick = () => authShow('authLogin');
const _e18 = $('#goSignup'); if(_e18) _e18.onclick = () => authShow('authSignup');
const _e19 = $('#vfBack'); if(_e19) _e19.onclick = () => { localStorage.removeItem

const _e20 = $('#btnSignup'); if(_e20) _e20.onclick = async () => {
  const email = $('#auEmail').value.trim().toLowerCase();
  const pseudo = $('#auPseudo').value.trim();
  const p1 = $('#auPass').value, p2 = $('#auPass2').value;
  if(!pseudo) return authErr('Choisis un pseudo.');
```

if(!/^[^\s@]+@[^\s@]+\.[^\s@]+\$/).test(email)) return authErr('Adresse email inv

if(p1.length < 8) return authErr('Mot de passe : 8 caractères minimum.');

if(p1 !== p2) return authErr('Les deux mots de passe ne correspondent pas.');

const u = { email, pseudo, hash: await sha(email + '::' + p1), verified:false,

const code = await issueCode(u);

\$('#vfEmail').textContent = email;

authShow('authVerify');

sendVerifyEmail(email, code);

```

};

const _e21 = $('#btnResend'); if(_e21) _e21.onclick = async () => {
  const u = getUser(); if(!u) return;
  const code = await issueCode(u);
  sendVerifyEmail(u.email, code);
};

const _e22 = $('#btnVerify'); if(_e22) _e22.onclick = async () => {
  const u = getUser(); if(!u) return;
  if(u.codeAt && Date.now() - u.codeAt > 15*60*1000)
    return authErr('Code expiré (15 min) - clique sur "Renvoyer le code".');
```

const input = \$('#vfCode').value.trim();

const okPlain = u.code && input === u.code; /* compat anciens comptes */

const okHash = u.codeH && (await sha('code::' + input)) === u.codeH;

if(!okPlain && !okHash){

u.tries = (u.tries||0) + 1; setUser(u);

if(u.tries >= 5){ u.codeH = null; u.code = null; setUser(u); return authErr('

return authErr(`Code incorrect (\${5 - u.tries} essai\${5-u.tries>1?'s':''} res

}

u.verified = true; delete u.code; delete u.codeH; delete u.tries; delete u.code

localStorage.setItem(LS_AUTH, '1');

authErr('');

enterApp();

toast('Compte vérifié - bienvenue ! 🎉');

```

};

const _e23 = $('#btnLogin'); if(_e23) _e23.onclick = async () => {

```

```

const u = getUser();
const email = $('#loEmail').value.trim().toLowerCase();
if(!u || u.email !== email) return authErr('Aucun compte avec cet email sur cet
if(await sha(email + '::' + $('#loPass').value) !== u.hash) return authErr('Mot
if(!u.verified){
  const code = await issueCode(u);
  $('#vfEmail').textContent = u.email; authShow('authVerify'); sendVerifyEmail(
  return;
}
localStorage.setItem(LS_AUTH, '1');
enterApp();
toast('Re-bonjour ' + email.split('@')[0] + ' 🙌');
};

```

```

function enterApp(){ $('#authWrap').classList.add('hidden'); renderProfile(); ren
function requireAuth(){
  const u = getUser();
  if(u && u.verified && localStorage.getItem(LS_AUTH) === '1'){ enterApp(); retur
  $('#authWrap').classList.remove('hidden');
  if(!u) authShow('authSignup');
  else if(!u.verified){ $('#vfEmail').textContent = u.email; authShow('authVerify
  else authShow('authLogin');
}

```

```

/* =====
CATÉGORIES — 🌍 Carte · 🗺️ Voyage · 👤 Profil
===== */
function switchCat(cat){
  $$('.catnav button').forEach(b => b.classList.toggle('on', b.dataset.cat === ca
  $('#catTrip').classList.toggle('hidden', cat !== 'trip');
  $('#catMap').classList.toggle('hidden', cat !== 'map');
  $('#catProfile').classList.toggle('hidden', cat !== 'profile');
  window.scrollTo({top:0});
  if(cat === 'map') buildProjectMap();
  if(cat === 'profile'){ renderProfile(); renderSettings(); }
}
$$('.catnav button').forEach(b => b.onclick = () => switchCat(b.dataset.cat));

```

```

/* =====
PRÉFÉRENCES — pilotent l'IA ET l'interface
===== */
const LS_SET = 'acolite_settings';
const SET_DEF = {
  style: [], /* détente, culture, aventure... (multi) */
  rythme: 'équilibre',
  food: 'aucun',
  acces: 'non',

```

```

eco: 'non',
model: 'auto',
detail: 'normal',
verif: true,          /* relecture croisée par une 2e IA */
reels: true,          /* données réelles (météo, trains, fériés...) */
font: 100,
motion: true          /* animations */
};

let SET = { ...SET_DEF };
function loadSettings(){
  try{ SET = { ...SET_DEF, ...(JSON.parse(localStorage.getItem(LS_SET)) || {}) };
    applySettings();
  }
function saveSettings(){
  try{ localStorage.setItem(LS_SET, JSON.stringify(SET)); }catch(e){}
  applySettings();
}
function applySettings(){
  document.documentElement.style.fontSize = (SET.font || 100) + '%';
  document.documentElement.classList.toggle('no-motion', !SET.motion);
}

/* Ce bloc part dans TOUS les prompts : l'IA connaît enfin tes goûts */
function prefsBlock(){
  const L = [];
  if(SET.style?.length) L.push(`Style de voyage recherché : ${SET.style.join(', '}`
  const R = { doux:'rythme DOUX : peu d\'activités par jour, du temps libre, pas
    equilibre:'rythme ÉQUILIBRÉ : 2-3 activités par jour',
    intense:'rythme INTENSE : programme dense, on optimise chaque heure
  L.push(R[SET.rythme] || R.equilibre);
  const F = { vege:'végétarien', vegan:'végan', halal:'halal', casher:'casher', s
  if(F[SET.food]) L.push(`Alimentation ${F[SET.food]} : les restaurants et adress
  if(SET.acces === 'oui') L.push("ACCESSIBILITÉ : le voyageur est à mobilité rédu
  if(SET.eco === 'oui') L.push("ÉCOLOGIE : le voyageur veut ÉVITER L'AVION quand
  const D = { court:'Sois CONCIS : phrases courtes, va à l\'essentiel.',
    normal:'', long:'Sois DÉTAILLÉ : explique tes choix, donne des astu
  if(D[SET.detail]) L.push(D[SET.detail]);
  return L.length ? `
  \nPRÉFÉRENCES PERMANENTES DU VOYAGEUR (à respecter dans TOUT
}

/* --- Rendu du panneau Préférences --- */
const OPT = {
  stStyle: { key:'style', multi:true, items:[['detente','🛖 Détente'],['cultur
  stRythme: { key:'rythme', items:[['doux','🐢 Doux'],['equilibre','⚖️ Équilibré'
  stFood: { key:'food', items:[['aucun','Aucune contrainte'],['vege','🥗 Végé
  stAcces: { key:'acces', items:[['non','Aucun besoin'],['oui','🚶 Mobilité réc
  stEco: { key:'eco', items:[['non','Peu importe'],['oui','🌱 Éviter l\'avi

```



```

    stIA:      { key:null,      toggles:[['verif','🔍 Relecture par une 2e IA'],['ree
    stUI:      { key:null,      toggles:[['motion','🌟 Animations']] }
  };
function renderSettings(){
  Object.entries(OPT).forEach(([id, cfg]) => {
    const box = $('#'+ id);
    if(!box) return;
    if(cfg.toggles){
      box.innerHTML = cfg.toggles.map([k, lbl]) =>
        `<div class="chip ${SET[k] ? 'on' : ''}" data-tog="${k}">${lbl} ${SET[k]
      return;
    }
    box.innerHTML = cfg.items.map([v, lbl]) => {
      const on = cfg.multi ? (SET[cfg.key] || []).includes(v) : SET[cfg.key] ===
      return `<div class="chip ${on ? 'on' : ''}" data-set="${cfg.key}" data-val=
    }).join('');
  });
  const m = $('#stModel'); if(m) m.value = SET.model;
  const dt = $('#stDetail'); if(dt) dt.value = SET.detail;
  const f = $('#stFont'); if(f) f.value = SET.font;
  const fv = $('#stFsVal'); if(fv) fv.textContent = SET.font + ' %';
}
document.addEventListener('click', e => {
  const c = e.target.closest('[data-set]');
  if(c){
    const k = c.dataset.set, v = c.dataset.val;
    if(c.dataset.multi === '1'){
      const arr = SET[k] || [];
      SET[k] = arr.includes(v) ? arr.filter(x => x !== v) : [...arr, v];
    } else SET[k] = v;
    saveSettings(); renderSettings();
    toast('✓ Préférence enregistrée – l\'IA en tiendra compte');
    return;
  }
  const tg = e.target.closest('[data-tog]');
  if(tg){
    SET[tg.dataset.tog] = !SET[tg.dataset.tog];
    saveSettings(); renderSettings();
    return;
  }
  if(e.target.id === 'stReset'){
    if(!confirm('Réinitialiser toutes tes préférences ?')) return;
    SET = { ...SET_DEF };
    saveSettings(); renderSettings();
    toast('🔄 Préférences réinitialisées');
  }
});

```

```

document.addEventListener('change', e => {
  if(e.target.id === 'stModel'){ SET.model = e.target.value; localStorage.removeItem
  if(e.target.id === 'stDetail'){ SET.detail = e.target.value; saveSettings(); }
});
document.addEventListener('input', e => {
  if(e.target.id !== 'stFont') return;
  SET.font = +e.target.value;
  $('#stFsVal').textContent = SET.font + ' %';
  saveSettings();
});

/* --- Barre "l'IA cherche" : remplace la nav du bas pendant la réflexion --- */
const SB_MSG = [
  'Acolite explore le monde...',
  'Il compare les destinations...',
  'Il vérifie les vols et les prix...',
  'Il repère les bons quartiers...',
  'Il finalise tes propositions...'
];
let _sbTimer = null;
function searchBar(on, first){
  const bar = $('#searchBar'), nav = $('.catnav');
  if(!bar || !nav) return;
  clearInterval(_sbTimer);
  if(on){
    let i = 0;
    $('#sbText').textContent = first || SB_MSG[0];
    bar.hidden = false;
    nav.style.display = 'none'; /* les 3 boutons laissent la place à la
    _sbTimer = setInterval(() => {
      i = (i + 1) % SB_MSG.length;
      $('#sbText').textContent = SB_MSG[i];
    }, 2200);
  } else {
    bar.hidden = true;
    nav.style.display = '';
  }
}

/* --- Carte du projet : sélecteur de trajet + carte plein cadre --- */
async function projRoute(route){
  const frame = $('#projMap');
  if(!frame) return;
  /* On centre la carte sur le point clé du trajet (OpenStreetMap : intégration l
  Google refuse l'affichage en iframe sans clé payante). */
  const cible = route.stops[route.stops.length - 1] || route.saddr;
  const key = 'geo_' + cible;

```

```

let g = state.cache[key];
if(!g){
  const r = await geoPlace(cible);
  if(r){ g = { lat:+r.latitude, lon:+r.longitude }; state.cache[key] = g; save(
}
if(!g){
  frame.src = 'https://www.openstreetmap.org/export/embed.html?bbox=-10,35,30,6
  return;
}
const dx = route.walk ? 0.012 : 0.06, dy = route.walk ? 0.008 : 0.04;
frame.src = `https://www.openstreetmap.org/export/embed.html`
+ `?bbox=${g.lon - dx},${g.lat - dy},${g.lon + dx},${g.lat + dy}`
+ `&layer=mapnik&marker=${g.lat},${g.lon}`;
}

function buildProjectMap(){
  const t = state.trip, p = state.prefs || {}, c = state.cache;
  const sel = $('#mapDay');
  if(!sel) return;
  if(!t){
    sel.innerHTML = '<option>Aucun voyage en cours</option>';
    sel.disabled = true;
    $('#projMap').src = 'https://www.openstreetmap.org/export/embed.html?bbox=-10
    if(navigator.geolocation){
      navigator.geolocation.getCurrentPosition(
        pos => {
          const la = pos.coords.latitude, lo = pos.coords.longitude;
          $('#projMap').src = `https://www.openstreetmap.org/export/embed.html?bb
        },
        () => {}, { timeout: 6000 }
      );
    }
    return;
  }
  sel.disabled = false;
  const routes = [];
  routes.push({ label:`$({plane:'✈️',train:'🚆',car:'🚗'})[state.mode]||'✈️'} A
    saddr: p.from || 'Paris', stops:['${t.nom}, ${t.pays}`], walk:false
  const base = c.plan?.logement?.quartier ? `${c.plan.logement.quartier}, ${t.nom
  const days = (c.plan?.programme || []).map(x => ({ jour:x.jour, resume:x.resume
  days.forEach(x => {
    const lieux = (x.lieux || []).filter(Boolean).map(l => `${l}, ${t.nom}`)
    if(lieux.length){
      routes.push({
        label: `📅 Jour ${x.jour} - ${String(x.resume || '').slice(0, 30)}`,
        saddr: base, stops: lieux, walk: true
      });
    }
  });

```

```

    }
  });
  window._projRoutes = routes;
  sel.innerHTML = routes.map((r, i) => `<option value="${i}">${esc(r.label)}</opt
  sel.value = '0';
  const r0 = routes[0];
  projRoute(r0);
  updateProjOpen(r0);
}
function updateProjOpen(r) {
  const a = $('#projOpen');
  if(a) a.href = 'https://www.google.com/maps/dir/' + [r.saddr, ...r.stops].map(e
}
document.addEventListener('change', e => {
  if(e.target.id !== 'mapDay') return;
  const r = (window._projRoutes || [])[+e.target.value];
  if(!r) return;
  projRoute(r);
  updateProjOpen(r);
});

/* --- Profil : infos + stats + paramètres --- */
function renderProfile(){
  const u = getUser(); if(!u) return;
  const pseudo = u.pseudo || u.email.split('@')[0];
  $('#pfAvatar').textContent = pseudo;
  $('#pfEmail').innerHTML = `${esc(pseudo)} <span style="cursor:pointer;font-size
  $('#pfMeta').innerHTML = `${esc(u.email)} · ${u.verified ? '✅ vérifié' : '⚠️ r
  const _e24 = $('#pfEditPseudo'); if(_e24) _e24.onclick = () => {
    const np = prompt('Ton nouveau pseudo :', pseudo);
    if(np && np.trim()){ u.pseudo = np.trim().slice(0,20); setUser(u); renderProf
  };
}
const _e25 = $('#pfExport'); if(_e25) _e25.onclick = () => $('#btnExport').click(
const _e26 = $('#pfNewTrip'); if(_e26) _e26.onclick = () => $('#btnReset').click(
const _e27 = $('#pfLogout'); if(_e27) _e27.onclick = () => {
  localStorage.removeItem(LS_AUTH);
  toast('À bientôt 🙌');
  requireAuth();
};
const LS_THEME = 'acolite_theme';
function applyTheme(){
  const dark = localStorage.getItem(LS_THEME) === 'dark';
  document.documentElement.dataset.theme = dark ? 'dark' : '';
  document.querySelectorAll('meta[name="theme-color"]').forEach(m => m.remove());
  const m = document.createElement('meta');
  m.name = 'theme-color';

```

```

    m.content = dark ? '#0B0B10' : '#FFE600';
    document.head.appendChild(m);
  }
const _e28 = $('#pfTheme'); if(_e28) _e28.onclick = () => {
  const cur = localStorage.getItem(LS_THEME) === 'dark' ? '' : 'dark';
  cur ? localStorage.setItem(LS_THEME, cur) : localStorage.removeItem(LS_THEME);
  applyTheme();
  toast(cur ? '🌙 Vol de nuit activé' : '☀️ Retour au jour');
};
applyTheme();

const _e29 = $('#pfChangePass'); if(_e29) _e29.onclick = async () => {
  const u = getUser(); if(!u) return;
  const cur = prompt('Mot de passe actuel :'); if(cur === null) return;
  if(await sha(u.email + '::' + cur) !== u.hash){ toast('❌ Mot de passe actuel i
const np = prompt('Nouveau mot de passe (8 caractères minimum) :'); if(np === n
if(np.length < 8){ toast('❌ 8 caractères minimum'); return; }
u.hash = await sha(u.email + '::' + np); setUser(u);
toast('🔑 Mot de passe changé ✔️');
};

const _e30 = $('#pfChangeEmail'); if(_e30) _e30.onclick = async () => {
  const u = getUser(); if(!u) return;
  const ne = (prompt('Nouvelle adresse email :') || '').trim().toLowerCase();
  if(!ne) return;
  if(!/^[^\s@]+@[^\s@]+\.[^\s@]+$/ .test(ne)){ toast('❌ Email invalide'); return;
  const pw = prompt('Confirme ton mot de passe :'); if(pw === null) return;
  if(await sha(u.email + '::' + pw) !== u.hash){ toast('❌ Mot de passe incorrect
u.email = ne;
u.hash = await sha(ne + '::' + pw);
u.verified = false;
const code = await issueCode(u);
localStorage.removeItem(LS_AUTH);
$('#vfEmail').textContent = ne;
sendVerifyEmail(ne, code);
requireAuth();
toast('✉️ Vérifie ta nouvelle adresse');
};

const _e31 = $('#pfClearCache'); if(_e31) _e31.onclick = () => {
  if(!confirm('Vider le cache IA ? Le voyage, tes notes et tes dépenses sont cons
state.cache = {}; save();
  toast('🧹 Cache IA vidé – contenus régénérés à la prochaine visite');
};

const _e32 = $('#pfMyData'); if(_e32) _e32.onclick = () => {
  const data = { compte: getUser(), voyage: state, exporte_le: new Date().toISOST

```

```

const blob = new Blob([JSON.stringify(data, null, 2)], {type:'application/json'}
const a = document.createElement('a');
a.href = URL.createObjectURL(blob);
a.download = 'acolite-mes-donnees.json';
a.click();
URL.revokeObjectURL(a.href);
toast('📄 Données téléchargées');
};

/* Suppression de compte : double confirmation (impossible par accident) */
const _e33 = $('#pfDelete'); if(_e33) _e33.onclick = () => {
  const u = getUser();
  const attendu = (u?.pseudo || 'SUPPRIMER');
  if(!confirm('⚠️ Dernière chance.\n\nTon compte, tes voyages, tes notes et tes s
const saisi = prompt(`Pour confirmer, écris ton pseudo exactement : \n\n${attend
if(saisi === null) return;
if(saisi.trim() !== attendu){
  toast('❌ Pseudo incorrect – suppression annulée');
  return;
}
['acolite_user', 'acolite_logged', 'acolite_trip_v2', 'acolite_settings', 'acolite_
  'acolite_theme', 'acolite_prices', 'acolite_docs', 'acolite_journal', 'acolite_cha
].forEach(k => localStorage.removeItem(k));
location.reload();
};

/* --- Filet de sécurité global : une erreur JS ne meurt plus en silence --- */
let _lastErrToast = 0;
window.addEventListener('error', () => {
  const now = Date.now();
  if(now - _lastErrToast > 8000){ _lastErrToast = now; try{ toast("⚠️ Oups, un pé
});
window.addEventListener('unhandledrejection', e => {
  const m = String(e.reason?.message||'');
  if(['NO_KEY', 'BAD_KEY', 'RATE', 'EMPTY', 'BAD_JSON', 'GROQ_RATE'].some(x=>m.include
  const now = Date.now();
  if(now - _lastErrToast > 8000){ _lastErrToast = now; try{ toast("⚠️ Une action
});

/* --- Voyage <-> QR : encodage compact + import --- */
function tripPayload(){
  const t = state.trip, p = state.prefs || {};
  if(!t) return null;
  const o = { v:1,
    trip: { nom:t.nom, pays:t.pays, drapeau:t.drapeau, iata:t.iata, budget_estime
    prefs: { from:p.from, days:p.days, when:p.when, depart:p.depart, adults:p.adu
  };

```

```

    return 'ACOLITE1:' + btoa(unescape(encodeURIComponent(JSON.stringify(o))));
}
function importPayload(str){
    if(!String(str).startsWith('ACOLITE1:')) throw new Error('format');
    const o = JSON.parse(decodeURIComponent(escape(atob(str.slice(9)))));
    if(!o.trip?.nom) throw new Error('vide');
    if(!confirm(`Importer le voyage "${o.trip.nom}, ${o.trip.pays}" ?\nTon voyage e
state.trip = o.trip;
state.prefs = { ...(state.prefs||{}), ...(o.prefs||{}) };
state.destinations = [o.trip];
state.cache = {}; state.checklist = {}; state.spend = []; state.notes = ''; st
state.planAnswers = []; state._qsDone = false; state.modeManual = false;
save(); unlockSteps();
switchCat('trip');
gotoStep(3);
toast(`📄 Voyage importé : cap sur ${o.trip.nom} !`);
return true;
}

/* --- Chargeurs de libs QR (cdnjs, à la demande, jamais bloquant) --- */
const _lib = {};
function loadLib(name, src, test){
    if(_lib[name]) return _lib[name];
    _lib[name] = new Promise((res, rej) => {
        if(test()) return res();
        const sc = document.createElement('script');
        const to = setTimeout(() => { _lib[name] = null; rej(new Error(name)); }, 800
        sc.src = src;
        sc.onload = () => { clearTimeout(to); test() ? res() : rej(new Error(name));
        sc.onerror = () => { clearTimeout(to); _lib[name] = null; rej(new Error(name)
        document.head.appendChild(sc);
    });
    return _lib[name];
}
const loadQRGen = () => loadLib('qrqgen', 'https://cdnjs.cloudflare.com/ajax/libs
const loadQRRead = () => loadLib('qrread', 'https://cdnjs.cloudflare.com/ajax/lib

/* --- Scanner : caméra ou photo -> import du voyage --- */
let _scanStream = null, _scanRun = false;
async function openScan(){
    $('#ovScan').classList.add('show');
    $('#scanMsg').textContent = '';
    try{ await loadQRRead(); }catch(e){ $('#scanMsg').textContent = '⚠️ Lecteur QR
    try{
        _scanStream = await navigator.mediaDevices.getUserMedia({ video: { facingMode
        const v = $('#scanVideo');
        v.srcObject = _scanStream; await v.play();

```

```

_scanRun = true;
const cv = document.createElement('canvas'), g = cv.getContext('2d');
(function tick(){
    if(!_scanRun) return;
    if(v.videoWidth && g){
        cv.width = v.videoWidth; cv.height = v.videoHeight;
        g.drawImage(v, 0, 0);
        const img = g.getImageData(0, 0, cv.width, cv.height);
        const q = window.jsQR(img.data, img.width, img.height);
        if(q && q.data.startsWith('ACOLITE1:')){
            closeScan();
            try{ importPayload(q.data); }catch(e){ toast('❌ QR illisible'); }
            return;
        }
    }
    requestAnimationFrame(tick);
})();
}catch(e){
    $('#scanMsg').textContent = '📷 Caméra indisponible ou refusée – choisis plut
}
}
function closeScan(){
    _scanRun = false;
    if(_scanStream){ _scanStream.getTracks().forEach(t => t.stop()); _scanStream =
    $('#ovScan').classList.remove('show');
}
const _cscanFile = $('#scanFile'); if(_cscanFile) _cscanFile.onchange = async e =
const f = e.target.files[0]; if(!f) return;
try{ await loadQRRead(); }catch(err){ $('#scanMsg').textContent = '⚠️ Lecteur Q
const img = new Image();
img.onload = () => {
    const cv = document.createElement('canvas'); cv.width = img.width; cv.height
    const g = cv.getContext('2d');
    if(!g){ $('#scanMsg').textContent = '❌ Lecture impossible.'; return; }
    g.drawImage(img, 0, 0);
    const q = window.jsQR(g.getImageData(0,0,cv.width,cv.height).data, cv.width,
    if(q && q.data.startsWith('ACOLITE1:')){ closeScan(); try{ importPayload(q.da
    else $('#scanMsg').textContent = '❌ Aucun QR Acolite détecté sur cette photc
};
img.src = URL.createObjectURL(f);
};
document.addEventListener('click', e => {
    if(e.target.id === 'btnScanTicket' || e.target.id === 'pfScan'){ openScan(); re
    if(e.target.closest('[data-closescan]')) closeScan();
});
}

/* --- Partage par lien : #v=payload → import direct à l'ouverture --- */

```



```

async function shareLink(){
  const pl = tripPayload();
  if(!pl){ toast('Choisis d'abord un voyage'); return; }
  const url = location.origin + location.pathname + '#v=' + encodeURIComponent(pl)
  const txt = `Mon voyage à ${state.trip.nom} sur Acolite ✈️`;
  try{
    if(navigator.share){ await navigator.share({ title:'Acolite', text:txt, url }
    await navigator.clipboard.writeText(url);
    toast('📋 Lien copié – envoie-le à tes amis');
  }catch(e){
    if(e.name !== 'AbortError') prompt('Copie ce lien :', url);
  }
}
document.addEventListener('click', e => { if(e.target.closest('[data-sharelink]')

/* import automatique si l'app est ouverte avec #v=... */
function checkImportHash(){
  const m = location.hash.match(/[#&]v=([^\&]+)/);
  if(!m) return;
  history.replaceState(null, '', location.pathname);
  try{ importPayload(decodeURIComponent(m[1])); }
  catch(e){ toast('❌ Lien de voyage invalide'); }
}

/* --- Export .ics : le programme dans ton agenda (Google/Apple/Outlook) --- */
function exportICS(){
  const t = state.trip, plan = state.cache.plan, d = stayDates();
  if(!t || !plan?.programme?.length || !d){ toast('Il faut un voyage avec une dat
  const pad = n => String(n).padStart(2, '0');
  const fmt = dt => `${dt.getUTCFullYear()}${pad(dt.getUTCMonth()+1)}${pad(dt.get
  const start = new Date(d.in + 'T00:00:00Z');
  const ev = (i, j) => {
    const day = new Date(start.getTime() + i * 86400000);
    const end = new Date(day.getTime() + 86400000);
    const lieux = (j.lieux||[]).join(', ');
    return ['BEGIN:VEVENT',
      `UID:acolite-${Date.now()}-${i}@acolite`,
      `DTSTAMP:${fmt(new Date())}T000000Z`,
      `DTSTART;VALUE=DATE:${fmt(day)}`,
      `DTEND;VALUE=DATE:${fmt(end)}`,
      `SUMMARY:J${j.jour} ${t.drapeau||''} ${String(j.resume||'').replace(/[,;\n]
      lieux ? `DESCRIPTION:${lieux.replace(/[,;\n]/g, ' ').slice(0,180)}` : '',
      `LOCATION:${String(t.nom).replace(/[,;\n]/g, ' ')}`,
      'END:VEVENT'].filter(Boolean).join('\r\n');
  };
const ics = ['BEGIN:VCALENDAR','VERSION:2.0','PRODID:-//Acolite//FR','CALSCALE:
...plan.programme.map((j, i) => ev(i, j)), 'END:VCALENDAR'].join('\r\n');

```

```

const a = document.createElement('a');
a.href = URL.createObjectURL(new Blob([ics], { type:'text/calendar' }));
a.download = `acolite-${String(t.nom).toLowerCase().replace(/[^a-z0-9]+/g, '-')}`;
a.click(); URL.revokeObjectURL(a.href);
toast('📅 Programme exporté – ouvre-le pour l’ajouter à ton agenda');
}
document.addEventListener('click', e => { if(e.target.closest('[data-ics]')) expo

/* --- Boarding pass → image PNG partageable (canvas maison) --- */
async function passPNG(){
  const t = state.trip, p = state.prefs || {};
  if(!t){ toast('Choisis d’abord un voyage'); return; }
  const W = 1200, H = 560, M = 30;
  const cv = document.createElement('canvas');
  cv.width = W; cv.height = H;
  const g = cv.getContext('2d');
  if(!g){ toast('Canvas indisponible'); return; }
  const K = '#101010', Y = '#FFE600', WH = '#FFFFFF', P = '#F4F3EF';
  const plan = state.cache.plan || {}, d = stayDates(), u = getUser();
  const CW = W - M * 2, CH = H - M * 2 - 12;          /* carte */
  const STUB = 300;                                   /* largeur du talon */
  const fit = (txt, size, max, weight = '700', fam = 'Inter, Arial') => {
    let px = size;
    do { g.font = `${weight} ${px}px ${fam}`; px -= 1; } while(g.measureText(txt)
    return g.font;
  };
  /* fond + ombre dure */
  g.fillStyle = P; g.fillRect(0, 0, W, H);
  g.fillStyle = K; g.fillRect(M + 12, M + 12, CW, CH);
  /* corps jaune + talon blanc */
  g.fillStyle = Y; g.fillRect(M, M, CW - STUB, CH);
  g.fillStyle = WH; g.fillRect(M + CW - STUB, M, STUB, CH);
  g.strokeStyle = K; g.lineWidth = 7; g.strokeRect(M, M, CW, CH);
  /* perforation entre corps et talon */
  const px0 = M + CW - STUB;
  g.setLineDash([12, 9]); g.lineWidth = 4;
  g.beginPath(); g.moveTo(px0, M + 26); g.lineTo(px0, M + CH - 26); g.stroke();
  g.setLineDash([]);
  g.fillStyle = P;
  [M, M + CH].forEach(cy => { g.beginPath(); g.arc(px0, cy, 15, 0, 7); g.fill();

  /* ---- CORPS ---- */
  g.fillStyle = K;
  g.font = '900 26px Sora, Arial';
  g.fillText('ACOLITE · BOARDING PASS', M + 34, M + 54);
  g.fillRect(M + 34, M + 66, 420, 5);
  /* route */

```

```

const from = (p.from || 'PAR').slice(0, 3).toUpperCase();
const to = (t.iata || t.nom.slice(0, 3)).toUpperCase();
g.font = '900 82px Sora, Arial';
g.fillText(from, M + 34, M + 158);
const wFrom = g.measureText(from).width;
const toX = M + 34 + wFrom + 190;
g.fillText(to, toX, M + 158);
/* trait pointillé + avion */
g.setLineDash([13, 9]); g.lineWidth = 5;
g.beginPath(); g.moveTo(M + 48 + wFrom, M + 132); g.lineTo(toX - 20, M + 132);
g.setLineDash([]);
g.font = '900 44px Arial';
g.fillText('✈', M + 48 + wFrom + (toX - 20 - (M + 48 + wFrom)) / 2 - 22, M + 14
/* infos, chacune redimensionnée pour ne jamais déborder */
const infoW = CW - STUB - 68;
const rows = [
  ['PASSAGER', `${(u?.pseudo || 'VOYAGEUR').toUpperCase()} · ${p.adults || 2} A`],
  ['DESTINATION', `${t.nom}, ${t.pays}`.toUpperCase()],
  ['DATES', d ? `${d.in} → ${d.out}` : (p.days || p.when || 'FLEXIBLES').toUpperCase()],
  ['SÉJOUR', [plan.transport?.mode, plan.logement ? String(plan.logement.type |
  ['BUDGET', plan.budget?.total ? `${plan.budget.total} € / PERSONNE` : (t.budg
];
rows.forEach((r, i) => {
  const y = M + 212 + i * 46;
  g.font = '800 15px Sora, Arial';
  g.fillText(r[0], M + 34, y);
  g.font = fit(r[1], 24, infoW - 130, '700');
  g.fillText(r[1], M + 164, y);
});
/* mentions bas de carte */
g.font = '600 14px Inter, Arial';
g.fillText("Ticket souvenir généré par Acolite — ne permet PAS d'embarquer ni d
g.fillText("Le QR sert uniquement à importer ce voyage dans l'application Acoli

/* ---- TALON : QR ---- */
const sx = px0 + STUB / 2;
let qrOK = false;
try{
  await loadQRGen();
  const tmp = document.createElement('div');
  new QRCode(tmp, { text: tripPayload(), width: 180, height: 180, correctLevel:
  await new Promise(r => setTimeout(r, 80));
  const q = tmp.querySelector('canvas') || tmp.querySelector('img');
  if(q){
    g.drawImage(q, sx - 90, M + 76, 180, 180);
    g.strokeStyle = K; g.lineWidth = 4;
    g.strokeRect(sx - 96, M + 70, 192, 192);

```

```

        qrOK = true;
    }
} catch(e) {}
if(!qrOK){
    g.fillStyle = K;
    let x = sx - 90; while(x < sx + 90){ const w2 = 4 + Math.floor(Math.random())
    }
    g.fillStyle = K;
    g.textAlign = 'center';
    g.font = '900 15px Sora, Arial';
    g.fillText('SCANNE-MOI', sx, M + 292);
    g.font = '800 12px Inter, Arial';
    g.fillText('dans l\'application Acolite', sx, M + 312);
    g.fillText('pour importer ce voyage', sx, M + 330);
    /* route + code-barres décoratif sur le talon */
    g.font = fit(`${from} ➔ ${to}`, 30, STUB - 50, '900', 'Sora, Arial');
    g.fillText(`${from} ➔ ${to}`, sx, M + 380);
    let bx = sx - 100;
    while(bx < sx + 100){ const w2 = 3 + Math.floor(Math.random() * 7); g.fillRect(
    g.textAlign = 'left';

cv.toBlob(async b => {
    const name = `acolite-${t.nom.toLowerCase().replace(/^[a-z0-9]+/g, '-')}.png`
    const file = typeof File !== 'undefined' ? new File([b], name, { type: 'image
    if(file && navigator.canShare?.({ files: [file] })){
        try{
            await navigator.share({ files: [file], title: 'Mon ticket Acolite', text:
            toast('🎫 Ticket partagé – le QR est scannable dans Acolite');
            return;
        } catch(e){ if(e.name === 'AbortError') return; }
    }
    const a = document.createElement('a');
    a.href = URL.createObjectURL(b);
    a.download = name;
    a.click();
    URL.revokeObjectURL(a.href);
    toast('📄 Ticket téléchargé – le QR est scannable dans Acolite');
}, 'image/png');
}
document.addEventListener('click', e => { if(e.target.closest('[data-passpng]'))

/* --- Ajuste la taille des valeurs pour qu'aucun mot ne soit coupé --- */
function fitStats(){
    $$('.plan-stat .v').forEach(el => {
        let px = 16;
        el.style.fontSize = px + 'px';
        while(el.scrollWidth > el.clientWidth + 1 && px > 9){

```

```

        px -= 0.5;
        el.style.fontSize = px + 'px';
    }
});
}

```

```

/* --- Météo animée en canvas (soleil / nuages / pluie / neige selon données réel
let _wxRun = 0;
function startWx(){
    const cv = $('#wxCv'), m = state.cache._real?.mNums;
    if(!cv || !m) return;
    const g = cv.getContext('2d');
    if(!g) return;
    const my = ++_wxRun;
    const mode = m.min <= 1 && m.rain > 25 ? 'snow' : m.rain > 55 ? 'rain' : m.rain
    const dark = () => document.documentElement.dataset.theme === 'dark';
    const drops = Array.from({length: 7}, (_, i) => ({ x: 8 + i * 6.5, y: Math.rand
    let f = 0;
    (function tick(){
        if(my !== _wxRun || !cv.isConnected) return;
        f++;
        g.clearRect(0, 0, 56, 56);
        const INK = dark() ? '#F4F3EF' : '#101010';
        if(mode === 'sun' || mode === 'cloud'){
            /* soleil qui tourne */
            const cx = mode === 'sun' ? 28 : 20, cy = mode === 'sun' ? 28 : 20, r = mod
            g.save(); g.translate(cx, cy); g.rotate(f * 0.02);
            g.strokeStyle = INK; g.lineWidth = 2.5;
            for(let i = 0; i < 8; i++){ g.rotate(Math.PI / 4); g.beginPath(); g.moveTo(
            g.restore();
            g.fillStyle = '#FFE600'; g.strokeStyle = INK; g.lineWidth = 2.5;
            g.beginPath(); g.arc(cx, cy, r, 0, 7); g.fill(); g.stroke();
        }
        if(mode !== 'sun'){
            /* nuage qui dérive */
            const ox = 6 * Math.sin(f * 0.03);
            g.fillStyle = dark() ? '#1B1B26' : '#fff'; g.strokeStyle = INK; g.lineWidth
            g.beginPath();
            g.arc(22 + ox, 32, 9, Math.PI * 0.5, Math.PI * 1.5);
            g.arc(30 + ox, 26, 8, Math.PI * 0.8, Math.PI * 1.98);
            g.arc(38 + ox, 32, 9, Math.PI * 1.5, Math.PI * 0.5);
            g.closePath(); g.fill(); g.stroke();
        }
        if(mode === 'rain' || mode === 'snow'){
            g.strokeStyle = mode === 'rain' ? '#00A8C0' : INK;
            g.fillStyle = g.strokeStyle; g.lineWidth = 2;
            drops.forEach(dp => {

```

```

    dp.y += mode === 'rain' ? 1.8 : 0.7;
    if(dp.y > 56) dp.y = 40;
    if(dp.y > 38){
        if(mode === 'rain'){ g.beginPath(); g.moveTo(dp.x, dp.y); g.lineTo(dp.x
        else { g.beginPath(); g.arc(dp.x + 2 * Math.sin(f * 0.1 + dp.x), dp.y,
        }
    }
    });
}
requestAnimationFrame(tick);
})();
}

```

```

/* --- Confettis 🎊 (valise complétée à 100 %) --- */
function confetti(){
    const cv = document.createElement('canvas');
    cv.style.cssText = 'position:fixed;inset:0;z-index:90;pointer-events:none';
    cv.width = innerWidth; cv.height = innerHeight;
    document.body.appendChild(cv);
    const g = cv.getContext('2d');
    if(!g){ cv.remove(); return; }
    const C = ['#FFE600', '#00F0FF', '#FF6B00', '#A855F7', '#22C55E', '#101010'];
    const ps = Array.from({length: 120}, () => ({
        x: Math.random() * cv.width, y: -20 - Math.random() * cv.height * 0.5,
        s: 6 + Math.random() * 8, v: 2.4 + Math.random() * 3.6,
        r: Math.random() * Math.PI, vr: (Math.random() - 0.5) * 0.25,
        c: C[Math.floor(Math.random() * C.length)]
    }));
    let n = 0;
    (function tick(){
        g.clearRect(0, 0, cv.width, cv.height);
        ps.forEach(q => {
            q.y += q.v; q.r += q.vr;
            g.save(); g.translate(q.x, q.y); g.rotate(q.r);
            g.fillStyle = q.c; g.fillRect(-q.s/2, -q.s/2, q.s, q.s);
            g.restore();
        });
        if(++n < 150) requestAnimationFrame(tick); else cv.remove();
    })();
}

```

```

/* --- PWA : app installable + hors-ligne --- */
if('serviceWorker' in navigator){
    navigator.serviceWorker.register('./sw.js').catch(()=>{});
}

```

```

/* bandeau hors-ligne (mode avion : l'app reste utilisable, l'IA non) */

```

```

function netBanner(){
  let b = $('#offBar');
  if(!navigator.onLine){
    if(!b){
      b = document.createElement('div');
      b.id = 'offBar';
      b.style.cssText = 'position:fixed;top:0;left:0;right:0;z-index:80;background-color:#f0f0f0;border:1px solid #ccc;';
      b.textContent = '✂ Hors-ligne – ton voyage reste consultable, l'IA et les autres services sont indisponibles';
      document.body.appendChild(b);
    }
  } else if(b) b.remove();
}

addEventListener('online', netBanner);
addEventListener('offline', netBanner);
netBanner();

/* bouton "Installer l'app" (Android/Chrome) – apparaît dans le Profil quand c'es
let _deferredPrompt = null;
addEventListener('beforeinstallprompt', e => {
  e.preventDefault();
  _deferredPrompt = e;
  const it = $('#pwaItem');
  if(it) it.style.display = '';
});

document.addEventListener('click', async e => {
  if(e.target.id !== 'pfInstall') return;
  if(!_deferredPrompt){ toast('Sur iPhone : Partager → « Sur l'écran d'accueil »' ); return; }
  _deferredPrompt.prompt();
  await _deferredPrompt.userChoice;
  _deferredPrompt = null;
  $('#pwaItem').style.display = 'none';
});

/* ----- Boot ----- */
loadSettings();
load();
checkImportHash();
if(state.prefs){
  $('#fFrom').value = state.prefs.from || '';
  $('#fWhen').value = state.prefs.when || '';
  if(state.prefs.depart) $('#fDepart').value = state.prefs.depart;
  if(state.prefs.dest) $('#fDest').value = state.prefs.dest;
  if(state.prefs.adults) $('#fAdults').value = state.prefs.adults;
  if(state.prefs.kids !== undefined) $('#fKids').value = state.prefs.kids;
  if(state.prefs.free) $('#fFree').value = (state.prefs.free||'').split(' | Affin
}
unlockSteps();

```

```
if(state.lastProps) renderDestinations(state.lastProps);  
if(state.step > 1) gotoStep(Math.min(state.step, 3));  
requireAuth();  
  
/* app.js est arrivé au bout : le vérificateur de démarrage ne déclenchera pas d'  
if(window.__ACOLITE) window.__ACOLITE.loaded = true;
```